

Program Studi Teknik Informatika
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung

Laporan Tugas Kecil 3 IF2211 Strategi Algoritma Ice Sliding Puzzle Solver



Nama Anggota 1 : Stevanus Agustaf Wongso
NIM : 13524020

Nama Anggota 2 : Vincent Rionarlie
NIM : 13524031

Semester II Tahun 2025/2026

Daftar Isi

1	Pendahuluan	3
1.1	Latar Belakang	3
1.2	Rumusan Masalah	3
1.3	Tujuan	3
1.4	Batasan Masalah	4
2	Dasar Teori dan Penjelasan Algoritma	5
2.1	Deskripsi Permasalahan	5
2.2	Representasi State	6
2.3	Mekanisme Pergerakan (<i>Sliding</i>)	7
2.4	Algoritma <i>Uniform Cost Search</i> (UCS)	8
2.5	Algoritma <i>Greedy Best-First Search</i> (GBFS)	9
2.6	Algoritma A^*	10
3	Analisis Algoritma	12
3.1	Definisi $f(n)$ dan $g(n)$ pada Setiap Algoritma	12
3.1.1	Uniform Cost Search (UCS)	12
3.1.2	Greedy Best-First Search (GBFS)	12
3.1.3	A^*	13
3.2	Fungsi Heuristik	13
3.2.1	H1 — Manhattan Distance ke Goal	13
3.2.2	H2 — Manhattan Distance Berbasis Checkpoint (<i>Primary</i>)	13
3.2.3	H3 — Estimasi Biaya Minimum	14
3.3	Admissibilitas Heuristik A^*	14
3.4	Perbandingan Teoritis Antar Algoritma	14
4	Implementasi	16
4.1	Lingkungan Pengembangan	16
4.2	Struktur Program	16
4.3	Antarmuka Grafis (GUI)	17
4.3.1	Mekanisme Save Output	18
4.4	Bagian Kunci Source Code	18
4.4.1	Fungsi <code>slide()</code>	18
4.4.2	Fungsi Heuristik (Manhattan)	19
4.4.3	Loop Utama A^*	19

5	Hasil Percobaan dan Analisis	21
5.1	Deskripsi Test Case	21
5.2	Hasil per Test Case	21
5.2.1	Test Case 1	21
5.2.2	Test Case 2	25
5.2.3	Test Case 3	29
5.2.4	Test Case 4	33
5.2.5	Test Case 5	37
5.3	Analisis Hasil	41
6	Kesimpulan	43
A	Tabel Checklist	46
B	Pranala Repository	47

Bab 1

Pendahuluan

1.1 Latar Belakang

Ice Sliding Puzzle adalah permainan grid di mana aktor bergerak di atas permukaan es yang licin. Setiap gerakan membuat aktor meluncur terus ke satu arah sampai terhenti oleh dinding, sehingga pemain tidak bisa berhenti di sembarang sel. Aturan ini membuat proses pencarian rute tidak sekadar masalah langkah terpendek, tetapi juga harus memperhitungkan urutan checkpoint, rintangan, lava, dan kondisi berhenti.

Dalam konteks Strategi Algoritma, masalah ini relevan karena dapat dimodelkan sebagai persoalan pencarian lintasan pada graf dengan biaya. Dibutuhkan pendekatan pencarian yang mampu mengelola biaya traversal dan prioritas ekspansi simpul. Oleh karena itu, proyek ini menerapkan UCS, GBFS, dan A* untuk membandingkan efektivitas algoritma uninformed dan informed dalam domain yang memiliki aturan pergerakan khusus.

1.2 Rumusan Masalah

Rumusan masalah utama adalah bagaimana merepresentasikan Ice Sliding Puzzle sebagai problem pencarian graf dengan state yang cukup untuk menggambarkan posisi aktor sekaligus progres checkpoint. Representasi ini harus mampu menangkap aturan meluncur, validasi urutan angka, serta kondisi menang dan kalah yang spesifik terhadap permainan.

Rumusan masalah berikutnya adalah bagaimana mengimplementasikan UCS, GBFS, dan A* untuk menemukan solusi, serta bagaimana membandingkan kinerja ketiganya dari sisi kualitas solusi, jumlah iterasi, dan waktu eksekusi. Selain itu, sistem perlu menyediakan visualisasi langkah demi langkah agar hasil pencarian dapat dianalisis secara lebih jelas.

1.3 Tujuan

Tujuan proyek ini tidak hanya menghasilkan solusi, tetapi juga menunjukkan perbedaan karakteristik tiga algoritma pencarian pada domain yang sama. Dengan visualisasi dan pencatatan metrik, perbandingan dapat dilakukan secara terukur.

Tujuan lain adalah menyediakan antarmuka yang memudahkan pengguna memilih input dan algoritma, serta meninjau jejak langkah solusi. Hal ini penting agar hasil eksperimen dapat direplikasi dan dilaporkan secara konsisten.

- Mengimplementasikan algoritma pathfinding UCS, GBFS, dan A* untuk menyelesaikan Ice Sliding Puzzle.
- Menganalisis dan membandingkan performa ketiga algoritma dari segi jumlah iterasi, cost solusi, dan waktu eksekusi.
- Memvisualisasikan proses pencarian solusi secara langkah per langkah.

1.4 Batasan Masalah

Batasan masalah ditetapkan agar implementasi tetap fokus pada spesifikasi tugas dan aturan permainan yang disepakati. Papan dibaca dari berkas teks dengan format yang ketat, sehingga validasi input dilakukan untuk memastikan ukuran, simbol, dan konten biaya sesuai.

Selain itu, batasan pada checkpoint dan simbol dibuat agar logika slide dapat ditentukan secara deterministik. Semua algoritma dieksekusi pada state yang sama dan menggunakan mekanisme pergerakan yang identik agar perbandingan performa bersifat adil.

- Program dikembangkan menggunakan bahasa C++.
- Papan permainan berukuran $N \times M$ dengan $N, M \geq 1$.
- Tile checkpoint hanya bernilai angka 0 hingga 9.
- Algoritma yang diimplementasikan: UCS, GBFS, dan A*.

Bab 2

Dasar Teori dan Penjelasan Algoritma

Pada bagian ini, konsep dasar pathfinding dirangkum dan diadaptasi ke konteks Ice Sliding Puzzle. Secara umum, masalah dapat dipandang sebagai *route planning* pada graf berbobot, dengan tujuan mencari lintasan berbiaya minimum dari state awal ke state tujuan.

Dalam teori pencarian, state space adalah himpunan semua konfigurasi yang mungkin, initial state adalah kondisi awal, goal test memeriksa apakah state saat ini mencapai tujuan, dan path cost adalah total biaya sepanjang lintasan. Definisi ini menjadi kerangka untuk menurunkan algoritma UCS, GBFS, dan A* pada puzzle.

2.1 Deskripsi Permasalahan

Secara konsep, Ice Sliding Puzzle adalah instans dari problem pencarian lintasan dengan graf berbobot. Setiap state merepresentasikan konfigurasi posisi aktor dan progres checkpoint, sedangkan edge merepresentasikan satu aksi sliding dengan biaya tertentu. Karena setiap aksi memiliki biaya non-negatif, problem ini cocok diselesaikan dengan algoritma pencarian biaya minimum seperti UCS dan A*.

Dalam kategori algoritma, UCS termasuk *uninformed search* karena tidak memanfaatkan informasi tambahan ke goal. GBFS dan A* termasuk *informed search* karena memanfaatkan heuristik $h(n)$ untuk memperkirakan biaya menuju goal. Perbedaan kategori ini menjelaskan perbedaan karakter eksplorasi di bagian implementasi dan hasil percobaan. Ice Sliding Puzzle direpresentasikan sebagai papan berukuran $N \times M$ berisi simbol-simbol tertentu. Aktor mulai dari posisi Z dan bergerak dengan mekanisme sliding, yaitu meluncur terus dalam satu arah sampai dihadang dinding X. Selama meluncur, aktor dapat melewati tile biasa *, checkpoint digit, atau tile tujuan 0.

Permainan dinyatakan berhasil jika aktor berhenti tepat di 0 setelah seluruh checkpoint digit 0 sampai digit terbesar dilalui secara berurutan. Pergerakan menjadi tidak valid apabila aktor keluar batas papan, melewati lava L, atau menginjak digit yang urutannya lebih besar dari checkpoint berikutnya.

Tabel 2.1: Keterangan Simbol pada Papan

Simbol	Keterangan
*	Tile kosong yang dapat dilewati
X	Rintangan/batu — aktor berhenti tepat sebelumnya
L	Lava — melewatinya menyebabkan <i>game over</i>
Z	Posisi awal aktor
0	Titik tujuan
0–9	Tile checkpoint — harus dilewati sesuai urutan angka



Gambar 2.1: Ilustrasi papan Ice Sliding Puzzle

2.2 Representasi State

Dalam teori, state harus memuat seluruh informasi yang memengaruhi transisi dan kondisi menang. Untuk Ice Sliding Puzzle, posisi saja tidak cukup karena status checkpoint menentukan langkah valid berikutnya. Oleh sebab itu, state didefinisikan sebagai (r, c, k) dengan k sebagai indeks checkpoint berikutnya.

Definisi ini membuat goal test dapat dirumuskan secara deterministik: state tujuan tercapai jika posisi aktor tepat di goal dan k melewati digit terbesar. Pada implementasi, state tersebut dipetakan ke struktur **State** dan digunakan sebagai kunci pada *visited set* maupun *best cost map*. State pada problem ini tidak hanya menyimpan posisi aktor, melainkan juga indeks checkpoint berikutnya yang harus diinjak. Ini penting karena

dua posisi yang sama bisa menghasilkan keputusan yang berbeda jika progres checkpoint berbeda. Implementasi menyatakan state sebagai pasangan posisi dan nilai `nextDigit`.

Dengan representasi ini, kondisi tujuan dapat diuji secara langsung: aktor harus berada di posisi 0 dan `nextDigit` melewati digit terbesar. Representasi tersebut juga memudahkan hashing dan penyimpanan state pada struktur *visited* atau *best cost*.

$$s = (r, c, k) \tag{2.1}$$

2.3 Mekanisme Pergerakan (*Sliding*)

Secara teori, operator pada problem pencarian mendefinisikan transisi antar state. Pada puzzle ini, operatornya adalah slide ke empat arah. Karena aktor meluncur hingga berhenti, satu aksi menghasilkan perpindahan multi-tile, sehingga biaya edge adalah penjumlahan biaya semua tile yang dilalui pada aksi tersebut.

Validitas transisi mengikuti aturan domain: keluar batas, melewati lava, atau melanggar urutan checkpoint membuat transisi tidak valid. Dengan demikian, setiap neighbor yang dihasilkan adalah state yang benar-benar dapat dicapai menurut aturan permainan. Pergerakan dilakukan dengan fungsi `slide` yang menerima arah (atas, bawah, kiri, kanan). Aktor bergerak selangkah demi selangkah hingga sel berikutnya adalah dinding X atau keluar batas papan. Jika keluar batas, gerakan dinyatakan gagal. Jika bertemu dinding, posisi berhenti adalah sel terakhir sebelum dinding.

Setiap sel yang dimasuki menambah biaya sesuai matriks biaya, dan digit checkpoint diperiksa berurutan selama proses meluncur. Bila digit yang ditemui lebih besar dari `nextDigit`, gerakan dianggap tidak valid. Selain itu, melewati lava L membatalkan gerakan sehingga state tujuan tidak ditambahkan ke frontier.

Algorithm 1 Fungsi SLIDE($board, costs, r, c, \Delta r, \Delta c, cleared$)

```
1:  $new\_cleared \leftarrow cleared$ 
2:  $slide\_cost \leftarrow 0$ 
3: while true do
4:    $(nr, nc) \leftarrow (r + \Delta r, c + \Delta c)$ 
5:   if  $(nr, nc)$  di luar batas papan then return INVALID
6:   end if
7:   if  $board[nr][nc] = X$  then return  $(r, c, new\_cleared, slide\_cost)$ 
8:   end if
9:   if  $board[nr][nc] = L$  then return INVALID
10:  end if
11:   $r, c \leftarrow nr, nc$ 
12:   $slide\_cost \leftarrow slide\_cost + costs[r][c]$ 
13:  if  $board[r][c]$  adalah digit  $d$  then
14:    if  $d \notin new\_cleared$  and  $d \neq |new\_cleared|$  then return INVALID  $\triangleright$  Urutan
    salah
15:    end if
16:    if  $d = |new\_cleared|$  then
17:       $new\_cleared \leftarrow new\_cleared \cup \{d\}$ 
18:    end if
19:  end if
20: end while
```

2.4 Algoritma *Uniform Cost Search* (UCS)

UCS berasal dari teori *uninformed search* dan menggunakan fungsi evaluasi $f(n) = g(n)$. Node yang di-expand selalu node dengan biaya aktual terkecil. Ketika goal pertama kali di-pop dari priority queue, biaya tersebut dijamin optimal selama semua biaya edge non-negatif.

Secara teoretis, UCS lengkap dan optimal, namun kompleksitasnya bisa tinggi karena mengeksplorasi banyak state tanpa panduan heuristik. Hal ini konsisten dengan perilaku yang diamati pada papan dengan ukuran besar. UCS pada proyek ini memperluas node berdasarkan biaya aktual paling kecil dari start ke state saat ini. Implementasi menggunakan priority queue dengan kunci g dan struktur `bestG` untuk menyimpan biaya terbaik per state. Pendekatan ini sejalan dengan Dijkstra karena selalu mengekskansi lintasan termurah yang diketahui.

Setiap kali node di-pop dari priority queue, jumlah iterasi ditambah. Bila state tersebut adalah goal, proses berhenti dan jalur direkonstruksi dari parent index. Karena seluruh edge cost bersifat non-negatif, UCS menjamin solusi optimal, namun cenderung melakukan eksplorasi yang lebih luas dibanding algoritma heuristik.

Algorithm 2 Algoritma UCS

```
1: Inisialisasi open_list dengan state awal,  $g = 0$ 
2: Inisialisasi closed_set kosong
3: while open_list tidak kosong do
4:    $node \leftarrow \text{pop node dengan } g \text{ terkecil dari } open\_list$ 
5:   if  $node.state \in closed\_set$  then
6:     continue
7:   end if
8:   Tambahkan  $node.state$  ke closed_set
9:   if  $node$  memenuhi kondisi menang then return solusi
10:  end if
11:  for setiap arah  $d \in \{U, D, L, R\}$  do
12:    Jalankan SLIDE dari  $node$ 
13:    if hasil valid then
14:      Push ke open_list dengan prioritas  $g_{baru}$ 
15:    end if
16:  end for
17: end while return Tidak ada solusi
```

2.5 Algoritma *Greedy Best-First Search* (GBFS)

GBFS termasuk *informed search* dengan fungsi evaluasi $f(n) = h(n)$. Algoritma ini memilih state yang tampak paling dekat ke goal berdasarkan heuristik, tanpa mempertimbangkan biaya aktual $g(n)$. Karena itu, GBFS tidak menjamin optimalitas meski sering cepat menemukan solusi.

Dalam teori, GBFS membutuhkan *visited set* agar tetap lengkap pada graf berhingga dan mencegah loop. Hal ini sesuai dengan implementasi yang menggunakan closed set untuk mencegah ekspansi ulang state yang sama. GBFS memilih node berdasarkan nilai heuristik terendah tanpa mempertimbangkan biaya aktual yang sudah ditempuh. Pada implementasi, priority queue berisi pasangan (h, idx) dan closed set digunakan untuk mencegah ekspansi ulang pada state yang sudah dikunjungi. Dengan strategi ini, GBFS lebih cepat menemukan jalur yang tampak menjanjikan.

Heuristik yang dipakai pada kode adalah jarak Manhattan dari posisi sekarang ke checkpoint berikutnya, atau ke goal jika semua checkpoint selesai. Akibat mengabaikan $g(n)$, GBFS tidak menjamin optimalitas dan berpotensi memilih jalur yang terlihat dekat tetapi berbiaya lebih besar.

Algorithm 3 Algoritma GBFS

```
1: Inisialisasi open_list dengan state awal, prioritas  $h(s_0)$ 
2: Inisialisasi closed_set kosong
3: while open_list tidak kosong do
4:    $node \leftarrow \text{pop node dengan } h \text{ terkecil dari } open\_list$ 
5:   if  $node.state \in closed\_set$  then
6:     continue
7:   end if
8:   Tambahkan  $node.state$  ke closed_set
9:   if  $node$  memenuhi kondisi menang then return solusi
10:  end if
11:  for setiap arah  $d \in \{U, D, L, R\}$  do
12:    Jalankan SLIDE dari  $node$ 
13:    if hasil valid then
14:      Push ke open_list dengan prioritas  $h(s_{baru})$ 
15:    end if
16:  end for
17: end while return Tidak ada solusi
```

2.6 Algoritma A^*

A^* menggabungkan biaya aktual dan estimasi biaya ke goal dengan $f(n) = g(n) + h(n)$. Secara teori, A^* optimal jika heuristik admissible (tidak pernah melebihi-lebihkan biaya sebenarnya). Jika heuristik juga consistent, A^* pada graph search tidak perlu membuka kembali node yang sudah diproses.

Dalam konteks puzzle ini, heuristik berbasis jarak Manhattan menjadi batas bawah jumlah tile yang harus dilalui, sehingga memenuhi sifat admissible selama biaya tile minimal bernilai satu. Hal ini menjelaskan mengapa A^* tetap optimal sekaligus lebih efisien daripada UCS. A^* menggabungkan biaya aktual $g(n)$ dan estimasi biaya ke goal $h(n)$ dengan fungsi evaluasi $f(n) = g(n) + h(n)$. Implementasi menggunakan priority queue dengan tie-breaking berdasarkan $-g$ dan urutan masuk agar ekspansi stabil. Struktur **bestG** menjaga agar state hanya diproses pada biaya terbaik.

Heuristik pada kode saat ini menggunakan jarak Manhattan langsung ke goal. Dengan g-map dan pemeriksaan biaya terbaik, A^* mempertahankan optimalitas sepanjang heuristik bersifat admissible. Secara empiris, A^* menyeimbangkan eksplorasi UCS yang menyeluruh dengan fokus heuristik seperti GBFS.

Algorithm 4 Algoritma A^*

```
1: Inisialisasi open_list dengan state awal,  $g = 0$ ,  $f = h(s_0)$ 
2: Inisialisasi closed_set dan g_map kosong
3:  $g\_map[s_0] \leftarrow 0$ 
4: while open_list tidak kosong do
5:    $node \leftarrow$  pop node dengan  $f$  terkecil
6:   if  $node.state \in closed\_set$  then
7:     continue
8:   end if
9:   Tambahkan  $node.state$  ke closed_set
10:  if  $node$  memenuhi kondisi menang then return solusi
11:  end if
12:  for setiap arah  $d \in \{U, D, L, R\}$  do
13:    Jalankan SLIDE dari  $node$ 
14:    if hasil valid then
15:       $g_{baru} \leftarrow node.g + slide\_cost$ 
16:      if  $g_{baru} < g\_map.get(s_{baru}, \infty)$  then
17:         $g\_map[s_{baru}] \leftarrow g_{baru}$ 
18:        Push ke open_list dengan prioritas  $f = g_{baru} + h(s_{baru})$ 
19:      end if
20:    end if
21:  end for
22: end while return Tidak ada solusi
```

Bab 3

Analisis Algoritma

3.1 Definisi $f(n)$ dan $g(n)$ pada Setiap Algoritma

Definisi $f(n)$ dan $g(n)$ merupakan inti perbandingan algoritma pencarian. Pada proyek ini, $g(n)$ selalu didefinisikan sebagai total biaya traversal semua tile yang dimasuki dari state awal hingga state n , sesuai akumulasi biaya pada fungsi sliding.

Perbedaan utama antar algoritma muncul pada bagaimana $f(n)$ dipakai sebagai prioritas. UCS hanya menggunakan $g(n)$, GBFS hanya menggunakan $h(n)$, sedangkan A^* menjumlahkan keduanya sehingga memperoleh keseimbangan antara biaya aktual dan estimasi ke goal.

3.1.1 Uniform Cost Search (UCS)

Pada UCS, fungsi evaluasi identik dengan biaya aktual yang telah ditempuh. Karena itu, prioritas frontier sepenuhnya ditentukan oleh $g(n)$ sehingga node dengan biaya termurah selalu diproses terlebih dahulu. Hal ini memberikan jaminan optimalitas pada graf berbobot non-negatif.

Dalam implementasi, nilai $g(n)$ disimpan di setiap node dan dibandingkan dengan **bestG** agar state tidak diekspansi dengan biaya lebih buruk. Dengan demikian, jalur yang dipilih selalu merupakan jalur termurah yang diketahui ke state tersebut.

$$f(n) = g(n) \quad (3.1)$$

$$g(n) = \sum_{i=1}^k cost(t_i) \quad (3.2)$$

3.1.2 Greedy Best-First Search (GBFS)

GBFS menggunakan $f(n) = h(n)$ sehingga prioritas murni ditentukan oleh estimasi kedekatan ke target. Dalam konteks puzzle ini, fungsi heuristik menggunakan jarak Manhattan ke checkpoint berikutnya atau goal ketika seluruh checkpoint telah dilewati.

Meskipun $g(n)$ tetap dihitung untuk pelaporan cost solusi, nilai ini tidak memengaruhi urutan ekspansi. Konsekuensinya, GBFS sering lebih cepat tetapi tidak menjamin solusi optimal karena bias ke arah target tanpa mempertimbangkan biaya aktual.

$$f(n) = h(n) \quad (3.3)$$

3.1.3 A^*

A^* mendefinisikan $f(n)$ sebagai penjumlahan $g(n)$ dan $h(n)$. Pada implementasi ini, $g(n)$ adalah akumulasi biaya slide, sedangkan $h(n)$ adalah jarak Manhattan ke goal. Kombinasi ini memberikan panduan heuristik tanpa mengabaikan biaya aktual.

Untuk menjaga optimalitas, A^* menyimpan **bestG** dan hanya mendorong state baru jika ditemukan biaya yang lebih kecil. Tie-breaking menggunakan $-g$ membuat pencarian lebih fokus pada jalur yang telah berkembang lebih dalam ketika f sama.

$$f(n) = g(n) + h(n) \quad (3.4)$$

3.2 Fungsi Heuristik

Fungsi heuristik dipakai oleh GBFS dan A^* untuk memperkirakan jarak dari state saat ini menuju tujuan. Heuristik yang baik membantu mengurangi eksplorasi yang tidak perlu, namun tidak boleh melebihi-lebihkan biaya jika ingin menjaga optimalitas A^* .

Dokumen desain ini mendefinisikan tiga heuristik (H1, H2, H3) untuk analisis teori. Pada implementasi saat ini, A^* menggunakan H1 dan GBFS memakai varian H1 yang mengarah ke checkpoint berikutnya, sedangkan H2 dan H3 dapat dipakai sebagai pengembangan lebih lanjut.

3.2.1 H1 — Manhattan Distance ke Goal

$$h_1(n) = |r_n - r_{\text{goal}}| + |c_n - c_{\text{goal}}| \quad (3.5)$$

H1 menghitung jarak Manhattan dari posisi saat ini ke goal. Heuristik ini sederhana, cepat dihitung, dan memberikan batas bawah jumlah tile yang perlu dilewati jika biaya minimum per tile bernilai satu.

Kelemahan H1 adalah tidak mempertimbangkan urutan checkpoint dan rintangan yang memaksa aktor meluncur. Karena itu, H1 cenderung terlalu optimistis dan kurang informatif pada papan yang kompleks.

3.2.2 H2 — Manhattan Distance Berbasis Checkpoint (*Primary*)

$$h_2(n) = d(n, cp_k) + \sum_{i=k}^{m-1} d(cp_i, cp_{i+1}) + d(cp_m, \text{goal}) \quad (3.6)$$

$$h_2(n) = d(n, \text{goal}) \quad (3.7)$$

H2 memperhitungkan jarak dari posisi saat ini ke checkpoint berikutnya, lalu berantai hingga goal. Dengan demikian, H2 mengintegrasikan aturan bahwa checkpoint harus diambil berurutan dan menghasilkan estimasi yang lebih relevan dibanding H1.

Heuristik ini lebih informatif karena tetap memberikan batas bawah jumlah tile yang harus dilewati, tetapi memasukkan biaya perjalanan antar checkpoint. Pada puzzle dengan banyak digit, H2 membantu prioritas eksplorasi menjadi lebih terarah.

3.2.3 H3 — Estimasi Biaya Minimum

$$h_3(n) = h_2(n) \times \text{cost_min} \quad (3.8)$$

H3 mengalikan H2 dengan biaya tile minimum pada papan. Tujuannya adalah mengubah estimasi jarak (dalam langkah Manhattan) menjadi estimasi biaya, sehingga lebih sesuai dengan fungsi evaluasi berbasis cost.

H3 lebih baik dari H2 ketika variasi biaya tile cukup besar dan cost minimum lebih besar dari satu. Pada kondisi tersebut, H3 dapat memberikan batas bawah biaya yang lebih ketat dibanding H2.

3.3 Admissibilitas Heuristik A^*

Heuristik dikatakan admissible apabila tidak pernah melebihi-lebihkan biaya optimal dari suatu state ke goal. Dengan asumsi semua biaya tile bernilai minimal 1, jarak Manhattan selalu menjadi batas bawah jumlah tile yang harus dilewati. Karena itu, H1 dan H2 tidak melampaui biaya sebenarnya dan tetap admissible dalam kerangka biaya non-negatif.

Untuk H2, aktor wajib melewati semua checkpoint berurutan, sehingga lintasan aktual minimal harus mencakup segmen-segmen Manhattan antar titik tersebut. H3 juga admissible karena mengalikan H2 dengan biaya minimum, sehingga masih berada di bawah atau sama dengan biaya sebenarnya. Dengan syarat ini, A^* yang memakai H1 tetap optimal, sementara H2 dan H3 menjadi pilihan heuristik yang lebih informatif bila diimplementasikan.

3.4 Perbandingan Teoritis Antar Algoritma

Tabel 3.1: Perbandingan Teoritis UCS, GBFS, dan A^*

Aspek	UCS	GBFS	A^*
Fungsi evaluasi	$g(n)$	$h(n)$	$g(n) + h(n)$
Menggunakan g	Ya	Tidak	Ya
Menggunakan h	Tidak	Ya	Ya
Optimal?	Ya	Tidak	Ya (jika h admissible)
Lengkap?	Ya	Ya*	Ya
Kompleksitas waktu	$O(b^{C^*/\varepsilon})$	$O(b^m)$	$O(b^d)$
Kompleksitas ruang	$O(b^{C^*/\varepsilon})$	$O(b^m)$	$O(b^d)$
Perilaku umum	Menyeluruh, lambat di board besar	Cepat, tidak optimal	Seimbang

Lengkap dengan visited set. b = branching factor, d = depth solusi optimal, m = max depth, C^ = optimal cost, ε = min edge cost.

Pada Ice Sliding Puzzle, UCS cenderung mengeksplorasi banyak state karena tidak memiliki panduan heuristik, sehingga waktu eksekusi meningkat pada papan besar. Sebaliknya, GBFS sangat cepat dalam menemukan solusi tetapi berisiko menghasilkan cost yang tidak optimal, terutama jika jalur tampak dekat ke goal namun mahal.

A^* berada di antara keduanya dengan memanfaatkan heuristik sekaligus biaya aktual. Dalam praktik, A^* biasanya menghasilkan solusi optimal dengan iterasi lebih sedikit dari UCS, namun tetap lebih stabil daripada GBFS dalam hal kualitas solusi. Perbedaan ini menjadi jelas ketika jumlah checkpoint meningkat atau papan bertambah besar.

Bab 4

Implementasi

4.1 Lingkungan Pengembangan

Pengembangan dilakukan dengan bahasa C++ menggunakan standar modern agar struktur data dan performa tetap efisien. Komponen GUI dibangun dengan raylib untuk menyediakan visualisasi yang ringan.

Tabel 4.1: Spesifikasi Lingkungan Pengembangan

Komponen	Keterangan
Bahasa Pemrograman	C++
Versi	GNU g++ (C++17)
Sistem Operasi	Unix-based

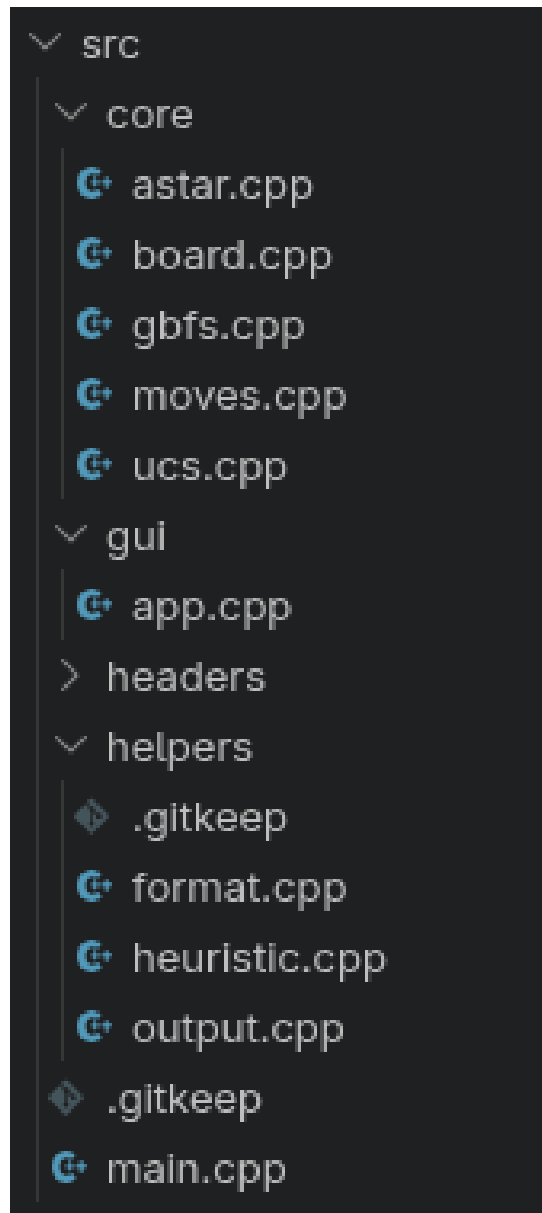
4.2 Struktur Program

Struktur program dibagi menjadi modul inti (core) untuk parsing, state, dan algoritma pencarian, serta modul GUI untuk visualisasi. Pemisahan ini memudahkan pengujian algoritma tanpa bergantung pada tampilan.

Berikut ringkasan struktur direktori dan peran file utama. Nama file disesuaikan dengan implementasi C++ yang digunakan pada proyek ini.

```
src/  
  main.cpp      - entry point, pemilihan mode GUI/CLI  
  core/  
    board.cpp   - parsing dan validasi input  
    moves.cpp   - aturan sliding dan generator move  
    ucs.cpp     - UCS solver  
    gbfs.cpp    - GBFS solver  
    astar.cpp   - A* solver  
  gui/  
    app.cpp     - implementasi GUI raylib  
  helpers/  
    format.cpp  - formatting output  
    output.cpp  - cetak solusi
```

headers/ - deklarasi struct dan fungsi

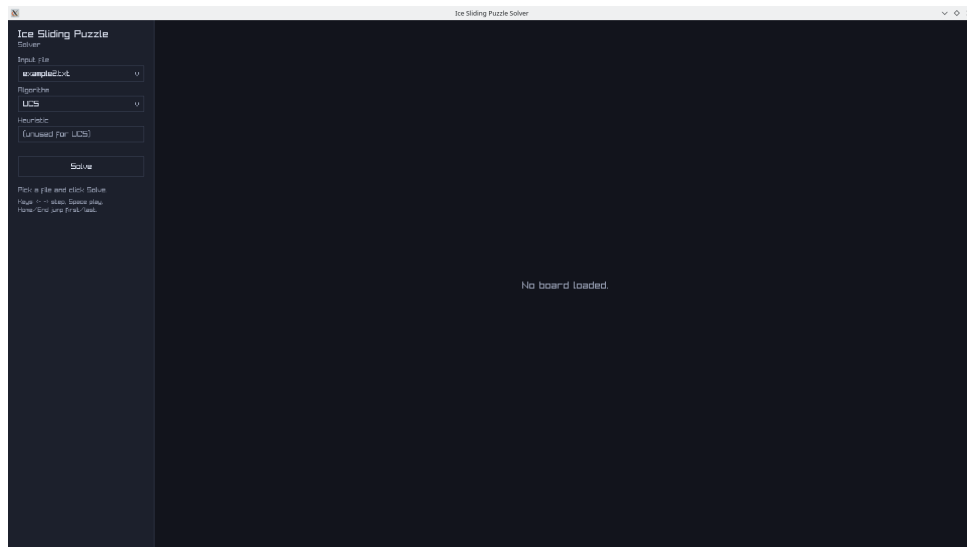


Gambar 4.1: Diagram struktur program (placeholder)

4.3 Antarmuka Grafis (GUI)

GUI berfungsi sebagai antarmuka utama untuk memilih input, algoritma, dan menampilkan hasil. Alur pengguna dimulai dari memilih file `.txt` pada folder `test/`, memilih algoritma (UCS, GBFS, A*), memilih heuristik untuk GBFS/A*, lalu menekan tombol **Solve** untuk menjalankan solver.

Setelah solusi ditemukan, panel kiri menampilkan ringkasan hasil (moves, cost, iterasi, waktu), dan panel kanan menampilkan papan beserta posisi aktor pada setiap langkah. Pengguna dapat memutar playback, mengubah kecepatan, melompat ke langkah tertentu melalui slider, serta menyimpan solusi ke berkas `solution_<input>.txt`.



Gambar 4.2: Tampilan GUI dan alur kontrol (placeholder)

4.3.1 Mekanisme Save Output

Program menyediakan fitur penyimpanan hasil solusi ke berkas teks secara otomatis. Setelah solusi ditemukan, pengguna dapat menekan tombol simpan di GUI, lalu program menulis ringkasan solusi beserta trace langkah ke folder `test/solution` sebagai lokasi default.

Berkas keluaran memuat informasi algoritma, heuristik, urutan gerakan, total cost, jumlah iterasi, dan waktu eksekusi, diikuti rekaman papan pada setiap langkah. Mekanisme ini memudahkan dokumentasi dan pengujian ulang tanpa perlu menjalankan program dari awal.

4.4 Bagian Kunci Source Code

Bagian ini menampilkan potongan kode inti yang menentukan perilaku puzzle dan algoritma pencarian. Fokus diberikan pada fungsi `slide`, fungsi heuristik, dan loop utama A* karena ketiganya mewakili aturan domain serta strategi pencarian yang digunakan.

4.4.1 Fungsi `slide()`

Fungsi `slide` bertanggung jawab mensimulasikan pergerakan aktor pada satu arah hingga berhenti. Implementasi mengecek batas papan, dinding, lava, serta urutan checkpoint, dan mengakumulasi biaya dari tiap tile yang dimasuki.

Potongan berikut diambil dari fungsi `SlideOne` pada modul `moves.cpp` dan menunjukkan logika utama selama proses meluncur.

```

1 // Potongan inti dari SlideOne (moves.cpp)
2 while (true) {
3     int nr = r + dr;
4     int nc = c + dc;
5
6     if (!InBounds(b, nr, nc)) return false; // keluar batas
7     char tile = b.grid[nr][nc];

```

```

8
9     if (tile == 'X') { // berhenti sebelum dinding
10         if (r == s.player.r && c == s.player.c) return false;
11         out.dir = dirChar;
12         out.next.player = {r, c};
13         out.next.nextDigit = curDigit;
14         out.cost = totalCost;
15         return true;
16     }
17
18     if (tile == 'L') return false; // lava
19     if (tile >= '0' && tile <= '9') {
20         int d = tile - '0';
21         if (d == curDigit) curDigit++;
22         else if (d > curDigit) return false;
23     }
24
25     totalCost += b.cost[nr][nc];
26     r = nr; c = nc;
27 }

```

Listing 4.1: Implementasi fungsi slide()

4.4.2 Fungsi Heuristik (Manhattan)

Heuristik pada implementasi GBFS memilih target berupa checkpoint berikutnya, sedangkan A* memakai jarak Manhattan ke goal. Pendekatan ini konsisten dengan desain bahwa GBFS bersifat greedy dan A* menjaga optimalitas dengan evaluasi $f(n) = g(n) + h(n)$.

Potongan berikut menunjukkan heuristik GBFS yang menghitung jarak Manhattan ke checkpoint berikutnya atau goal.

```

1 static int Heuristic(const Board& b, const State& s) {
2     Pos target;
3     if (s.nextDigit <= b.maxDigit) target = b.digits[s.nextDigit];
4     else target = b.goal;
5     return abs(s.player.r - target.r) + abs(s.player.c - target.c);
6 }

```

Listing 4.2: Implementasi heuristik Manhattan pada GBFS

4.4.3 Loop Utama A*

Loop utama A* mengekskansi node dengan nilai f terkecil, memeriksa goal, lalu menghasilkan tetangga dari hasil sliding. Struktur `bestG` memastikan hanya jalur termurah yang dipertimbangkan untuk state yang sama.

Cuplikan berikut menunjukkan bagian inti dari loop A* yang mengelola priority queue, best cost, dan ekspansi node.

```

1 while (!pq.empty()) {
2     PQItem it = pq.top(); pq.pop();

```

```

3      sol.iterations++;
4
5      const State cur = pool[it.idx].s;
6      auto bg = bestG.find(cur);
7      if (bg != bestG.end() && pool[it.idx].g > bg->second) continue;
8
9      if (IsGoal(b, cur)) { goalIdx = it.idx; break; }
10
11     for (const Move& mv : GenerateMoves(b, cur)) {
12         int ng = pool[it.idx].g + mv.cost;
13         auto it2 = bestG.find(mv.next);
14         if (it2 != bestG.end() && ng >= it2->second) continue;
15         bestG[mv.next] = ng;
16         pool.push_back({mv.next, ng, it.idx, mv.dir});
17         int nh = Heuristic(b, mv.next);
18         int nf = ng + nh;
19         pq.push({nf, -ng, pushId++, (int)pool.size() - 1});
20     }
21 }

```

Listing 4.3: Loop utama algoritma A*

Bab 5

Hasil Percobaan dan Analisis

5.1 Deskripsi Test Case

Lima test case dipilih dari folder `test/` dengan format nama `tc<angka>_<ket>.txt`. Keempat test case pertama memiliki ukuran 10x10 agar perbandingan antar algoritma stabil, sedangkan test case kelima berukuran 12x12 untuk menguji skala yang sedikit lebih besar.

Setiap test case tetap mengikuti aturan checkpoint berurutan. Satu pengecualian ada pada TC-2 yang tidak memiliki lava, sehingga perilaku sliding dapat diamati tanpa penalti tile L.

Tabel 5.1: Deskripsi Test Case

TC	Ukuran	Checkpoint	Lava	Keterangan
1	10 × 10	1	Ya	tc1_simple.txt
2	10 × 10	3	Tidak	tc2_digits.txt
3	10 × 10	2	Ya	tc3_lava.txt
4	10 × 10	4	Ya	tc4_costtrap.txt
5	12 × 12	5	Ya	tc5_killing.txt

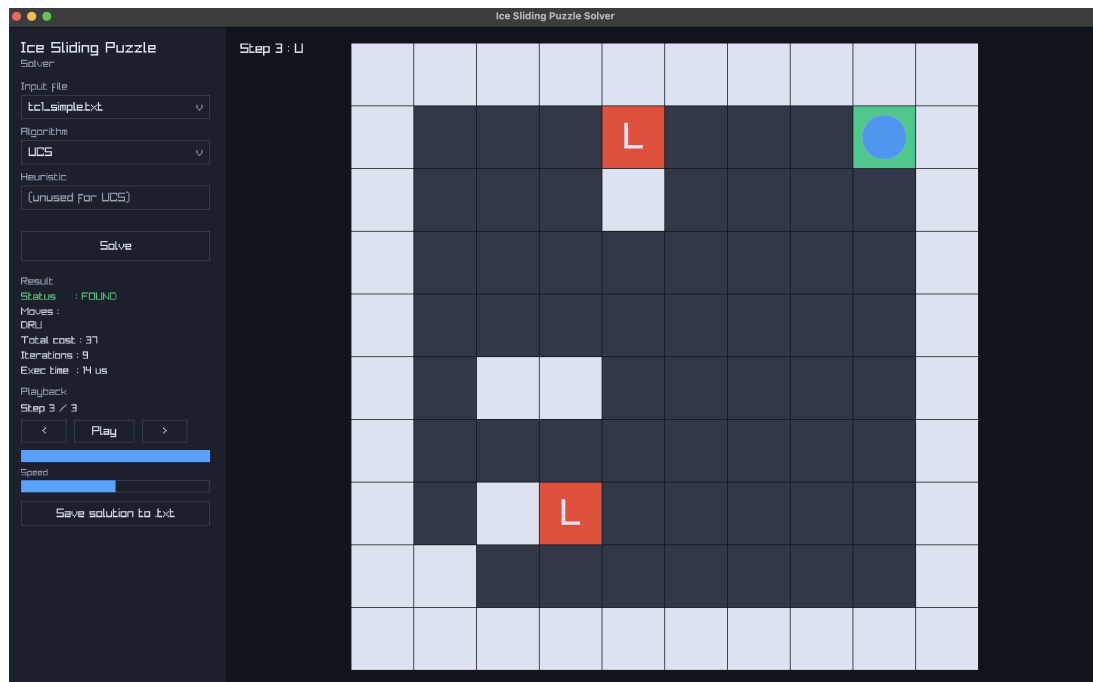
5.2 Hasil per Test Case

Bagian ini menyajikan ringkasan hasil eksekusi untuk setiap test case. Untuk setiap test case, ditampilkan papan input, screenshot output untuk tujuh variasi algoritma, dan tabel ringkas berisi solusi, cost, dan iterasi.

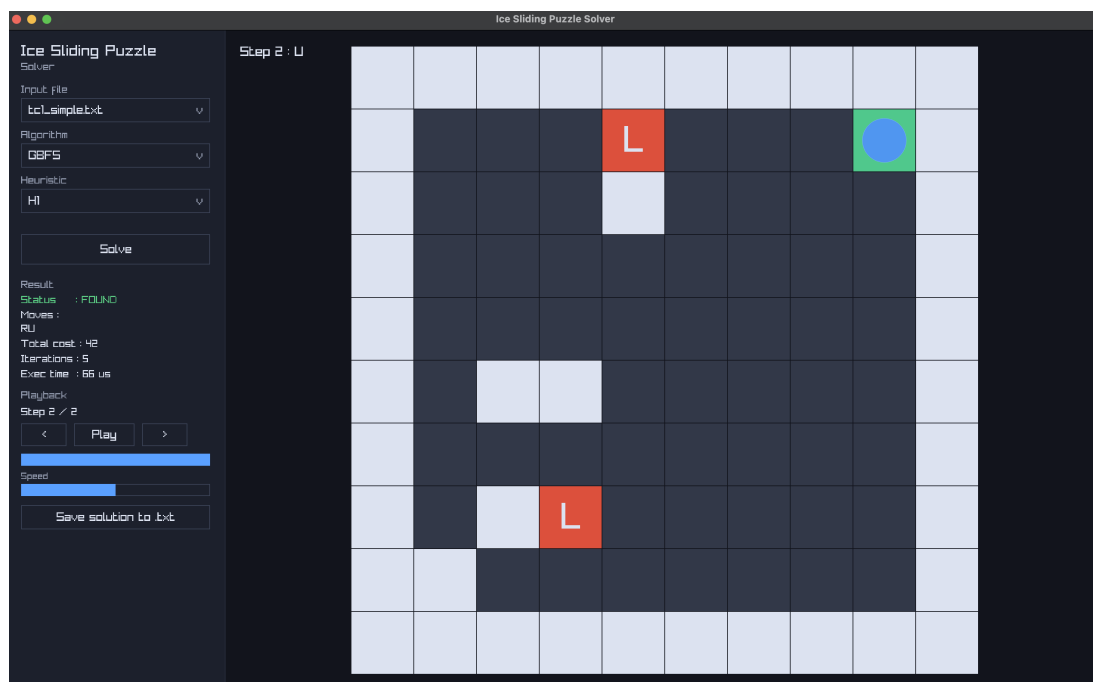
5.2.1 Test Case 1

Test case ini menggunakan file `test/tc1_simple.txt` berukuran 10x10. Fokusnya adalah memastikan aturan sliding, lava, dan urutan digit berjalan sesuai spesifikasi pada papan kecil.

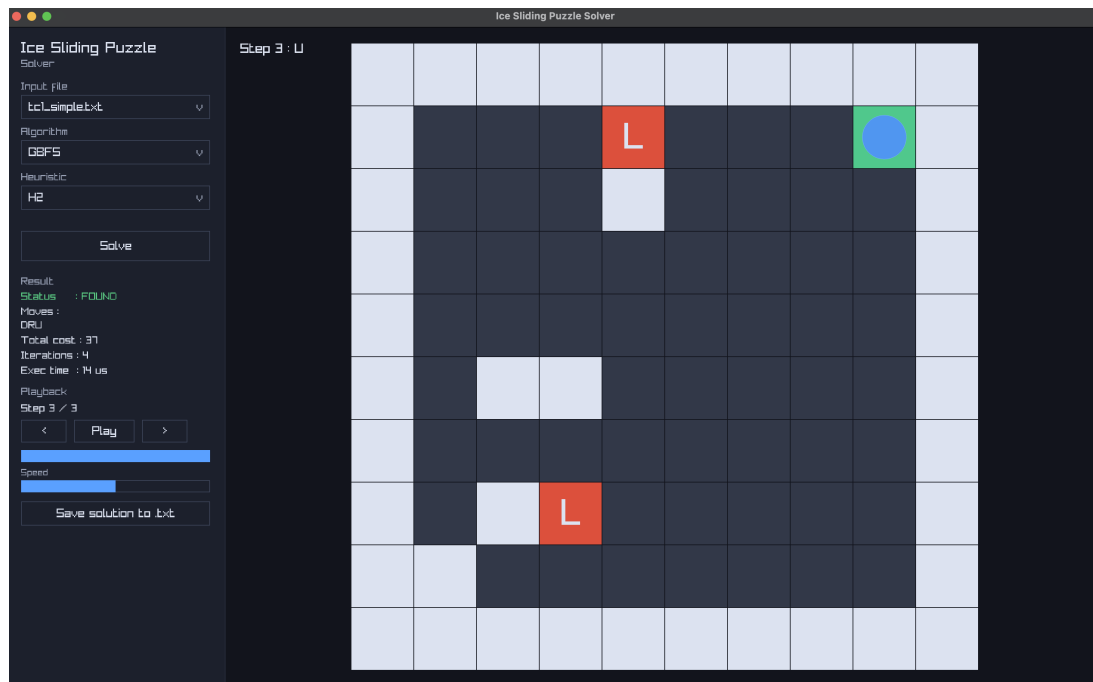
Hasil dari tujuh variasi algoritma pada papan ini digunakan sebagai baseline sebelum beralih ke papan yang lebih kompleks.



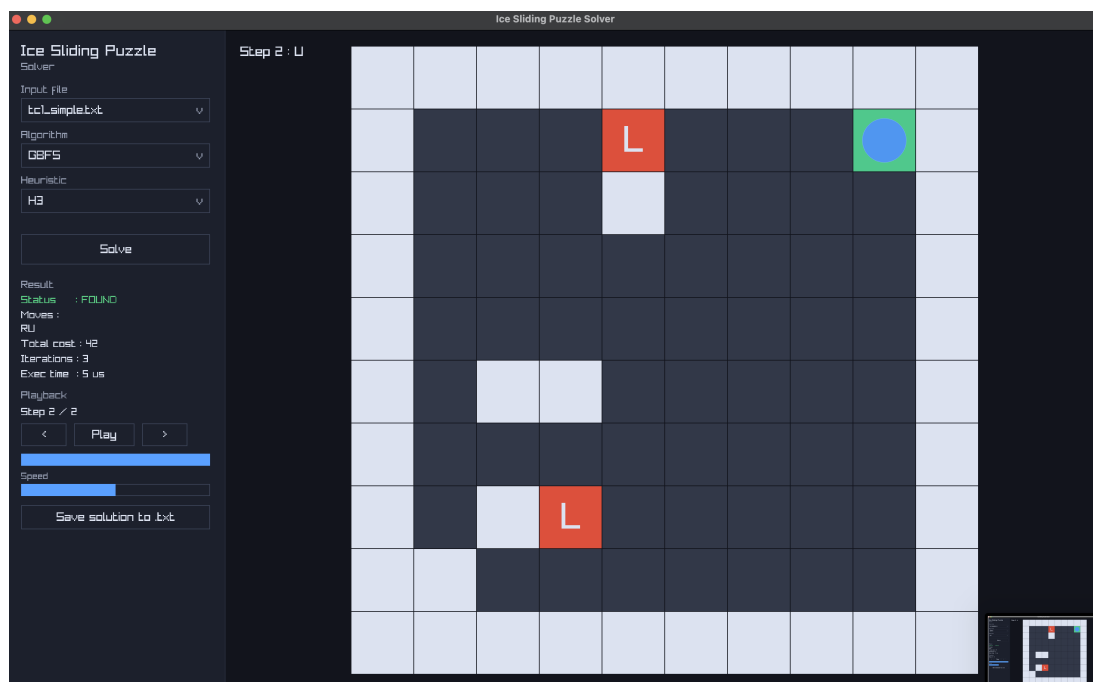
Gambar 5.1: UCS



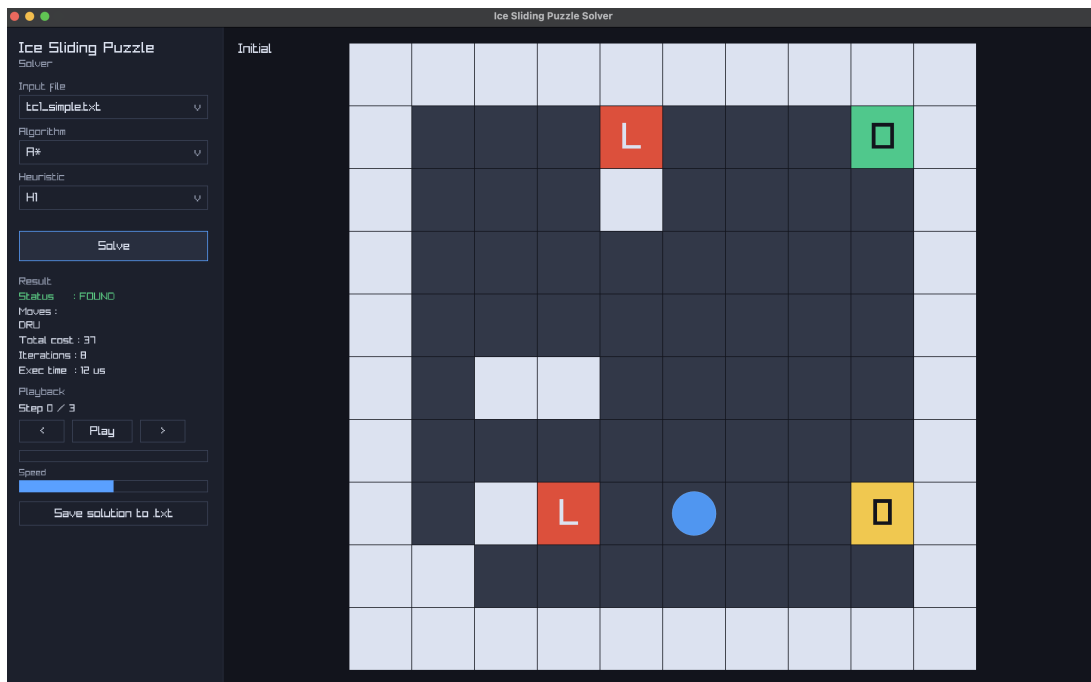
Gambar 5.2: GBFS H1



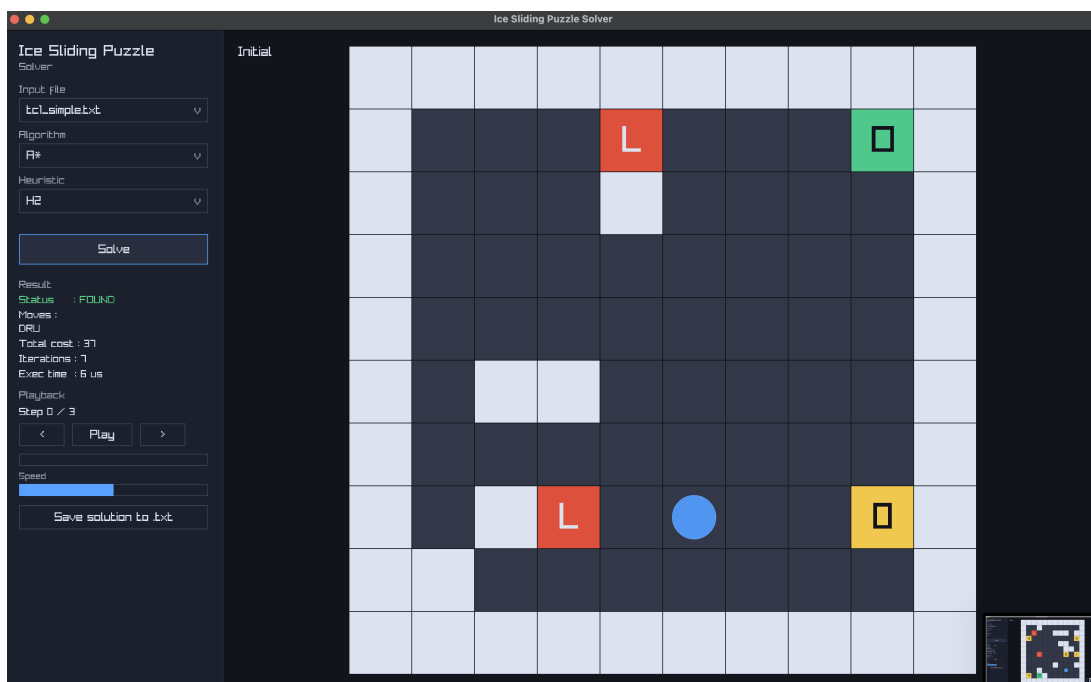
Gambar 5.3: GBFS H2



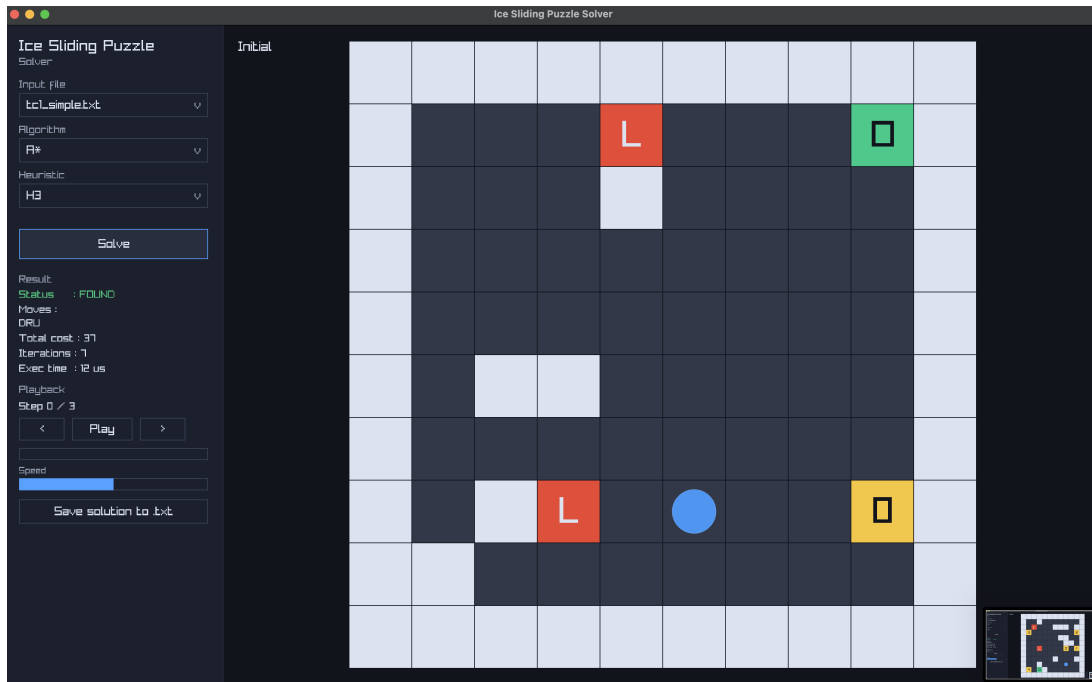
Gambar 5.4: GBFS H3



Gambar 5.5: A* H1



Gambar 5.6: A* H2



Gambar 5.7: A* H3

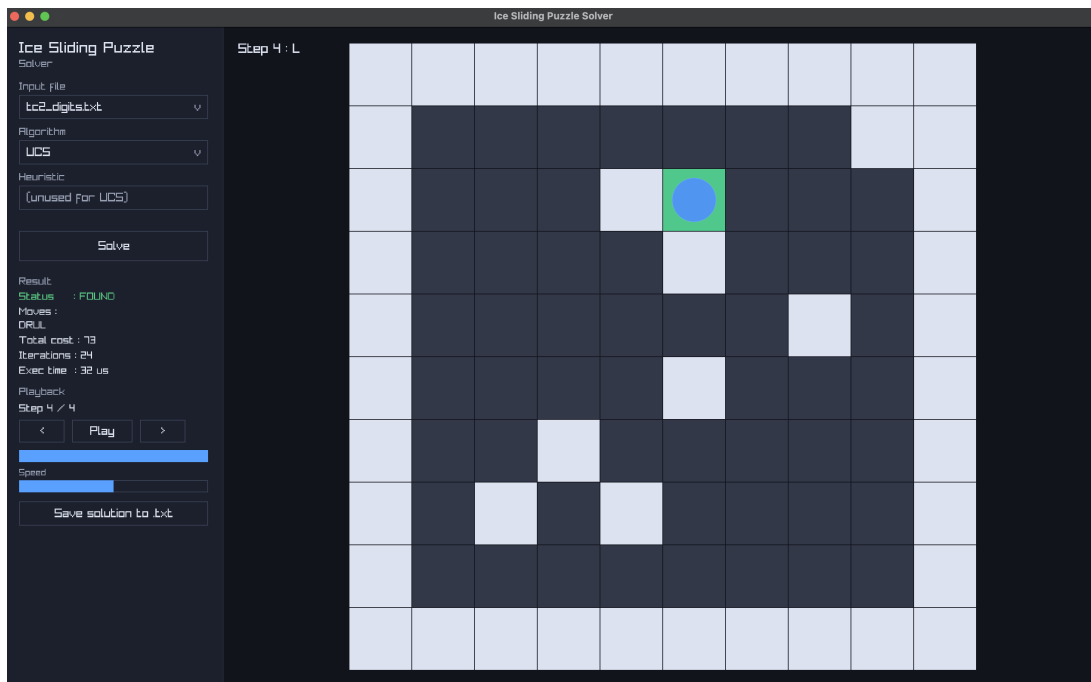
Tabel 5.2: Hasil TC-1

Algoritma	Solusi	Cost	Iterasi
UCS	DRU	37	9
GBFS H1	RU	42	5
GBFS H2	DRU	37	4
GBFS H3	RU	42	3
A* H1	DRU	37	8
A* H2	DRU	37	7
A* H3	DRU	37	7

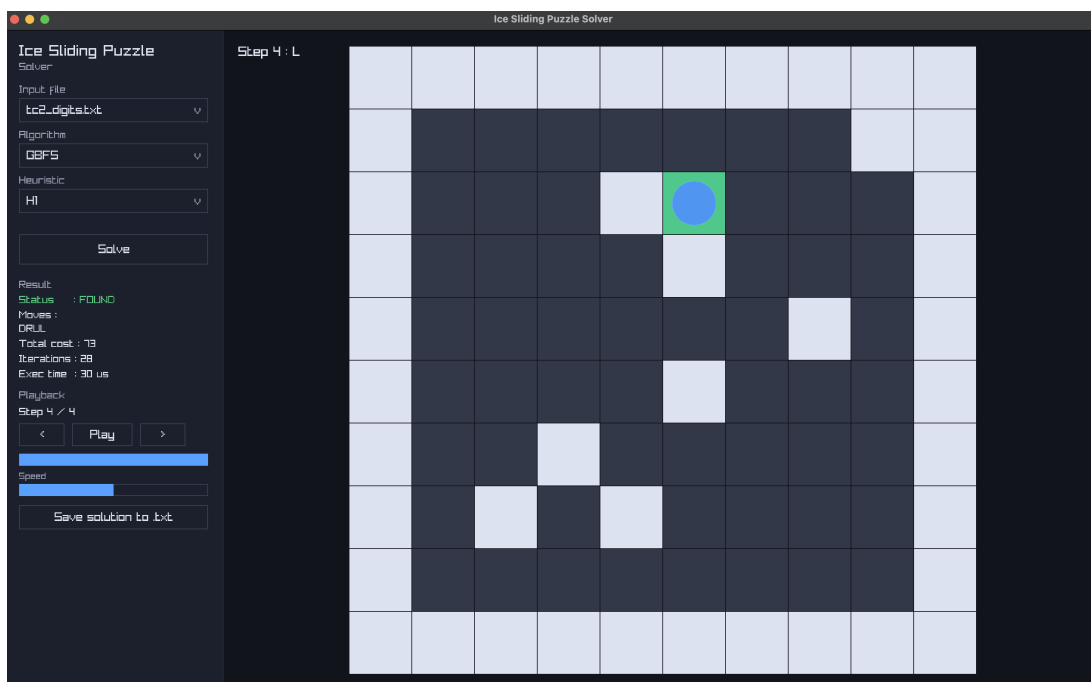
5.2.2 Test Case 2

Test case ini menggunakan file `test/tc2_digits.txt` berukuran 10x10. Papan ini tidak memiliki lava, sehingga fokus pengujian ada pada urutan checkpoint dan biaya traversal.

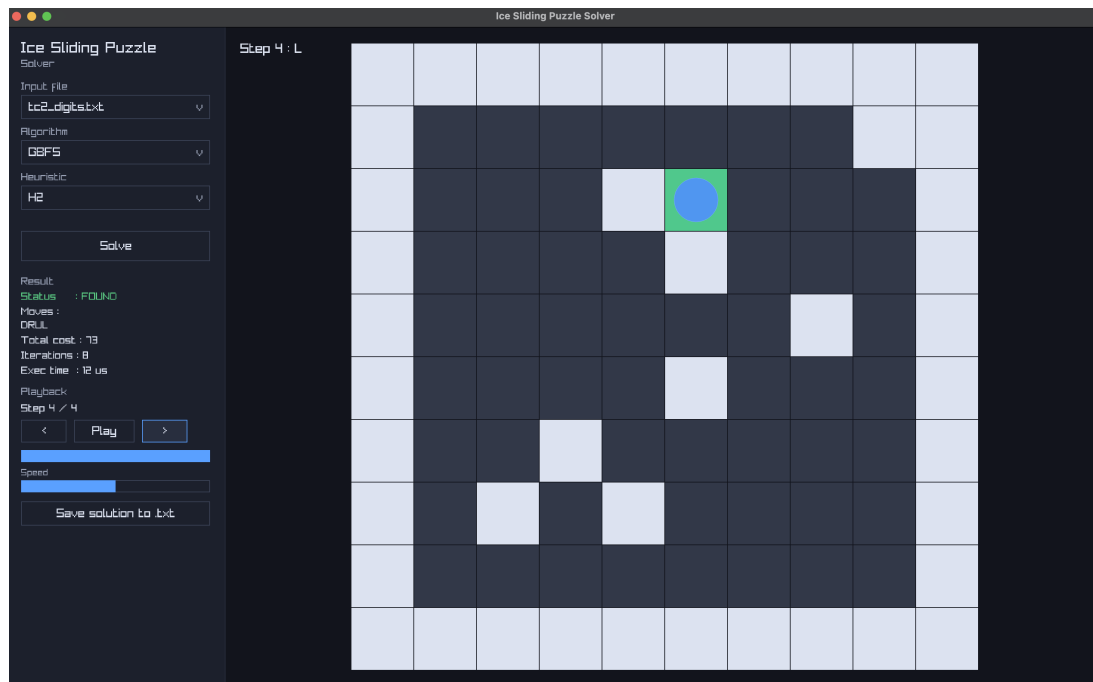
Perbandingan hasil pada TC-2 membantu menilai stabilitas algoritma terhadap variasi tata letak tanpa penalti tile L.



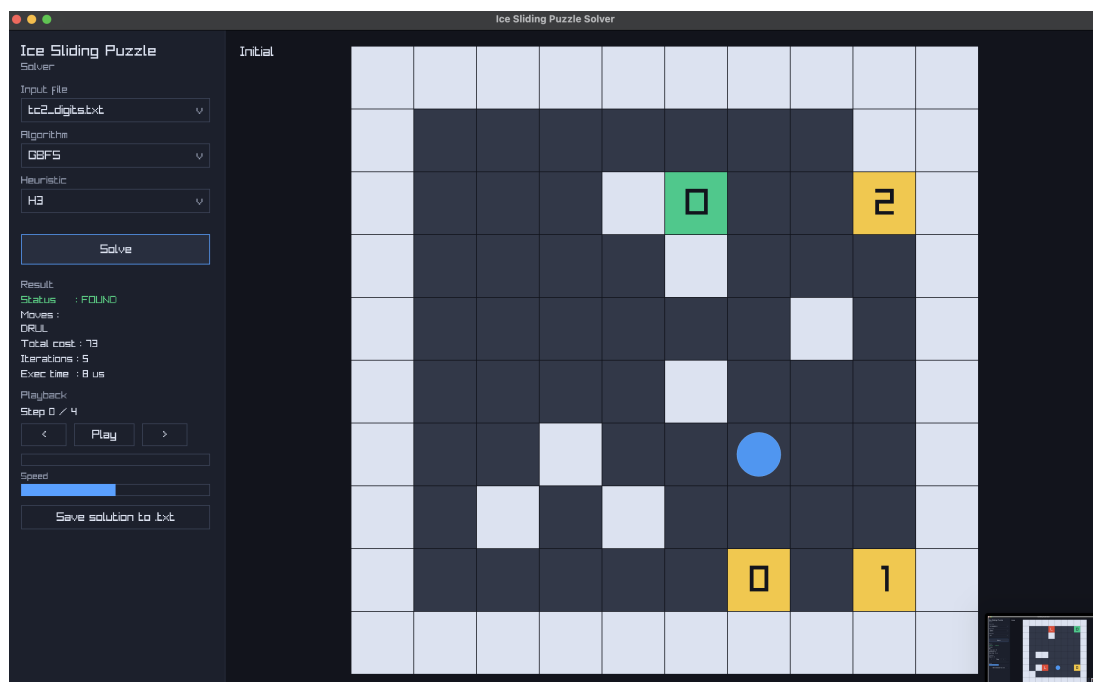
Gambar 5.8: UCS



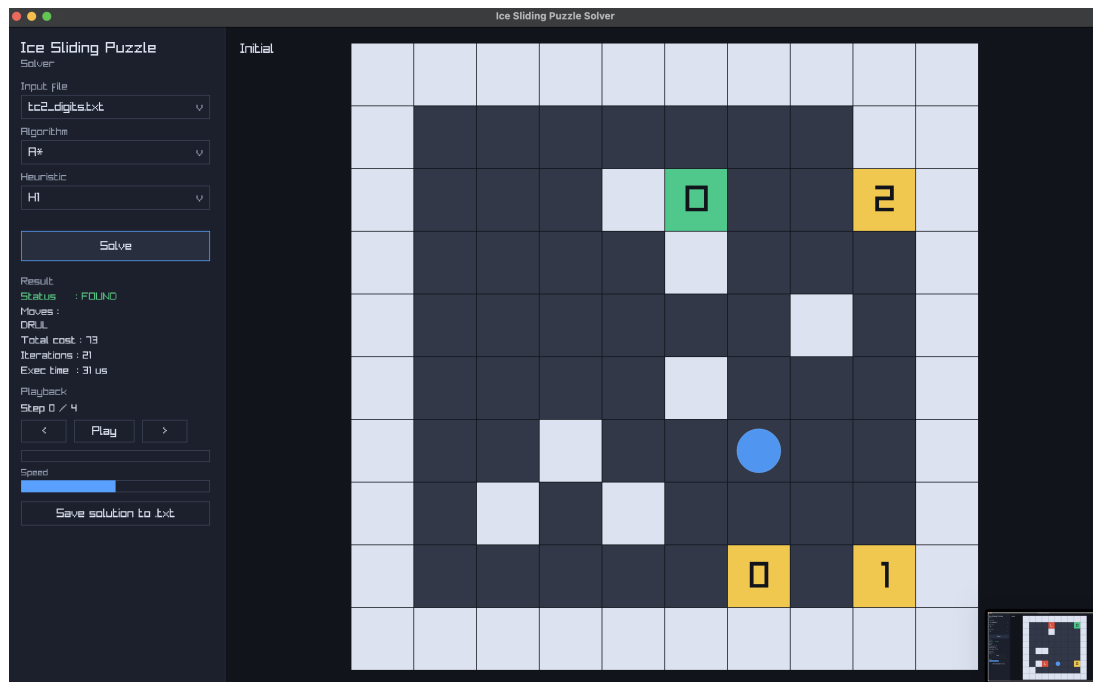
Gambar 5.9: GBFS H1



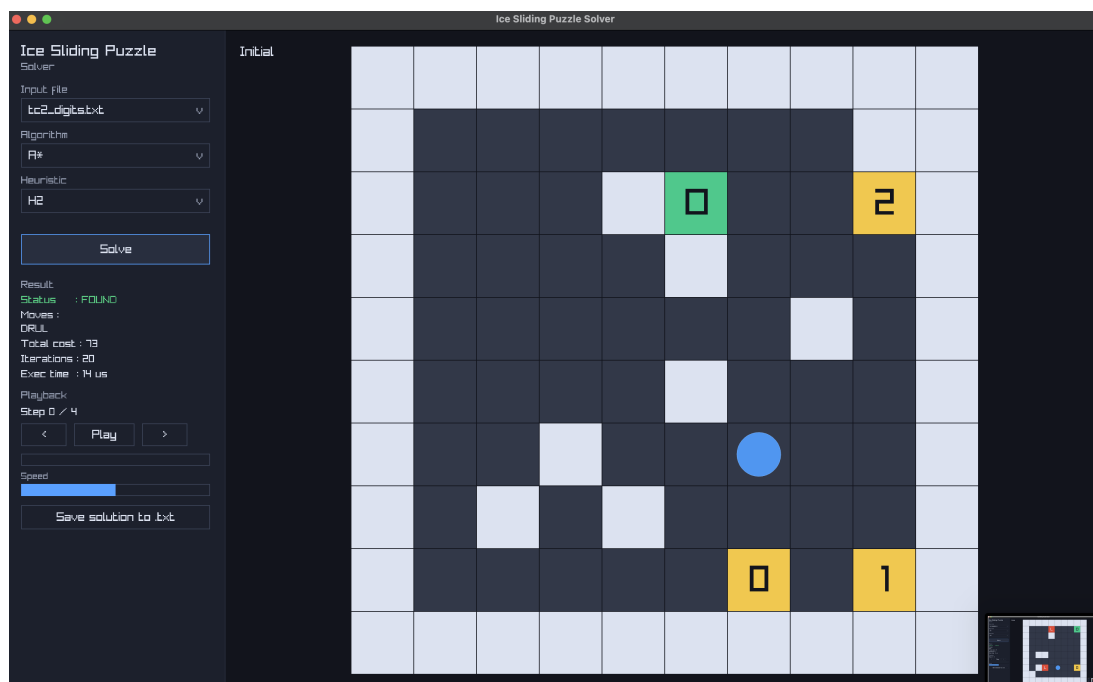
Gambar 5.10: GBFS H2



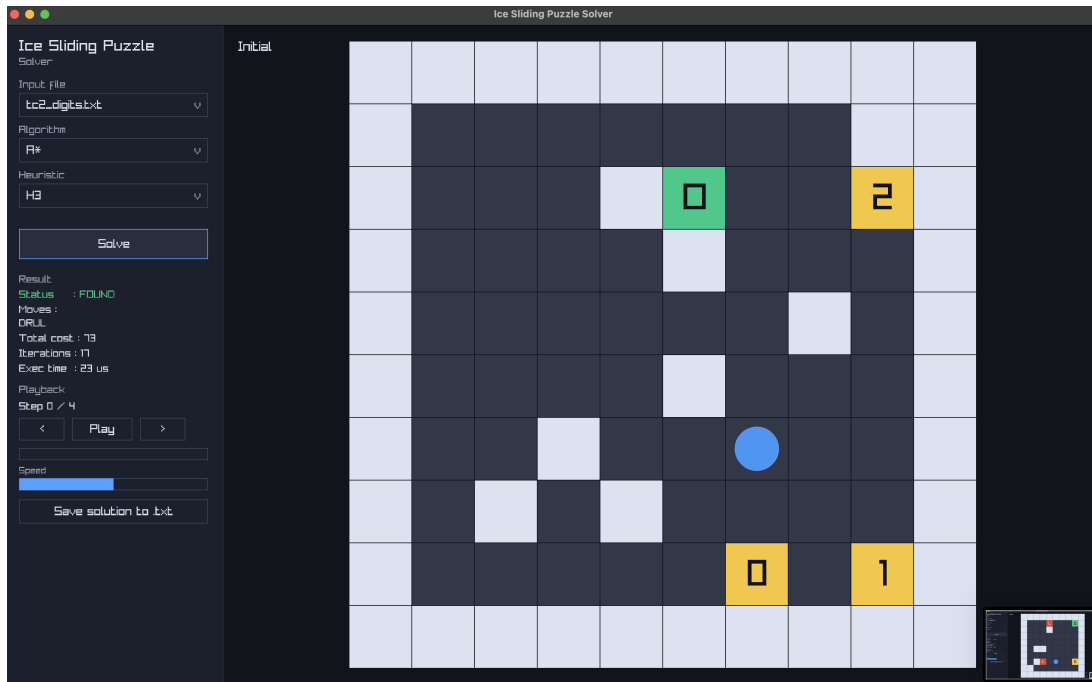
Gambar 5.11: GBFS H3



Gambar 5.12: A* H1



Gambar 5.13: A* H2



Gambar 5.14: A* H3

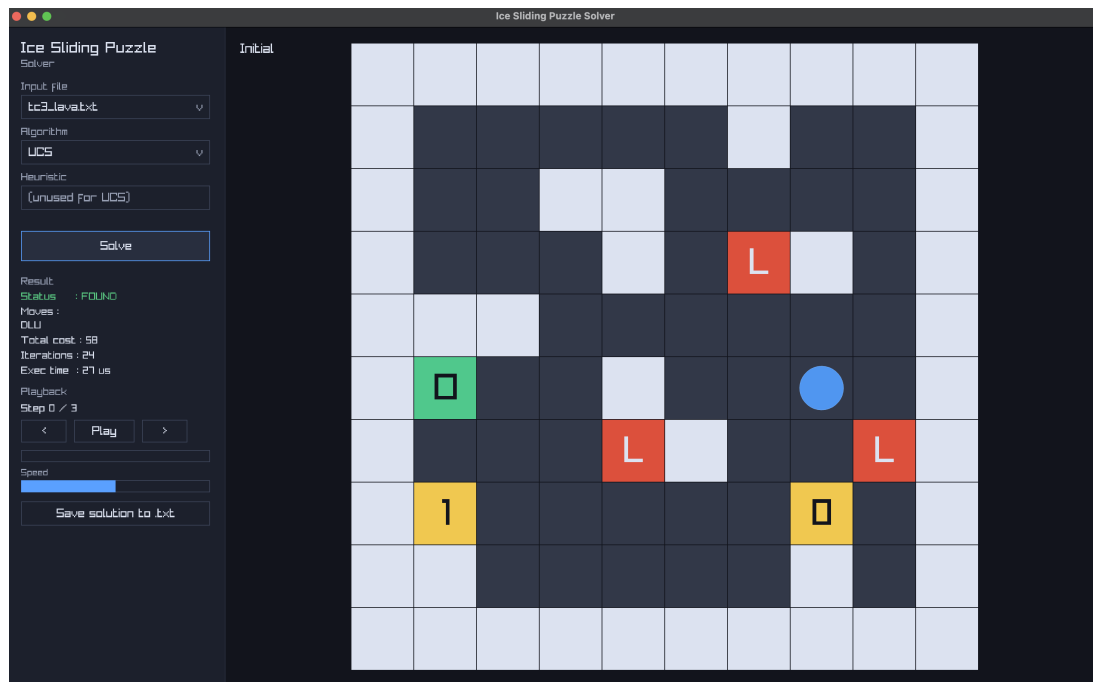
Tabel 5.3: Hasil TC-2

Algoritma	Solusi	Cost	Iterasi
UCS	DRUL	73	24
GBFS H1	DRUL	73	28
GBFS H2	DRUL	73	8
GBFS H3	DRUL	73	5
A* H1	DRUL	73	21
A* H2	DRUL	73	20
A* H3	DRUL	73	17

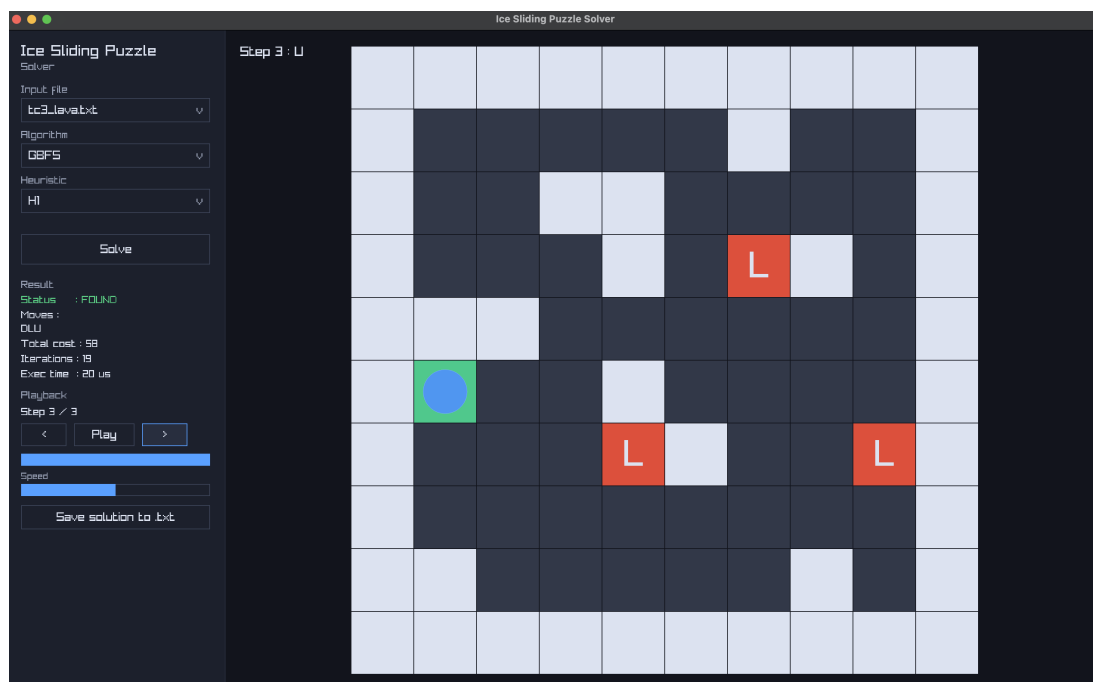
5.2.3 Test Case 3

Test case ini menggunakan file `test/tc3_lava.txt` berukuran 10x10. Papan ini memuat lava sehingga validasi gerak menjadi lebih ketat.

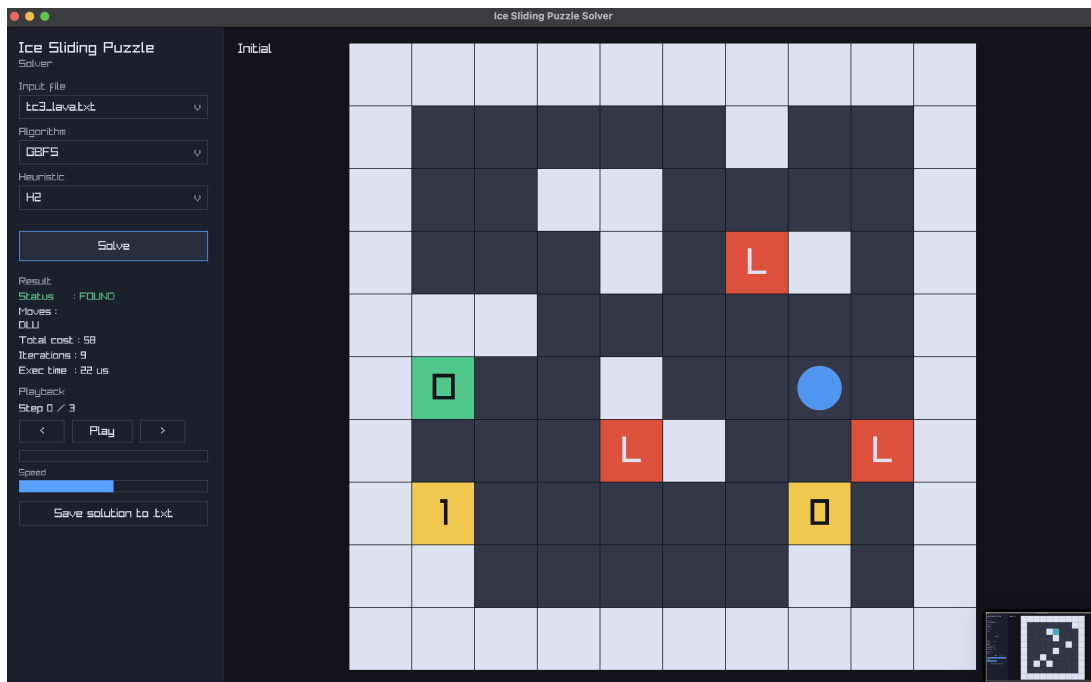
Hasil pada TC-3 menunjukkan dampak tile L terhadap jumlah iterasi dan kualitas solusi.



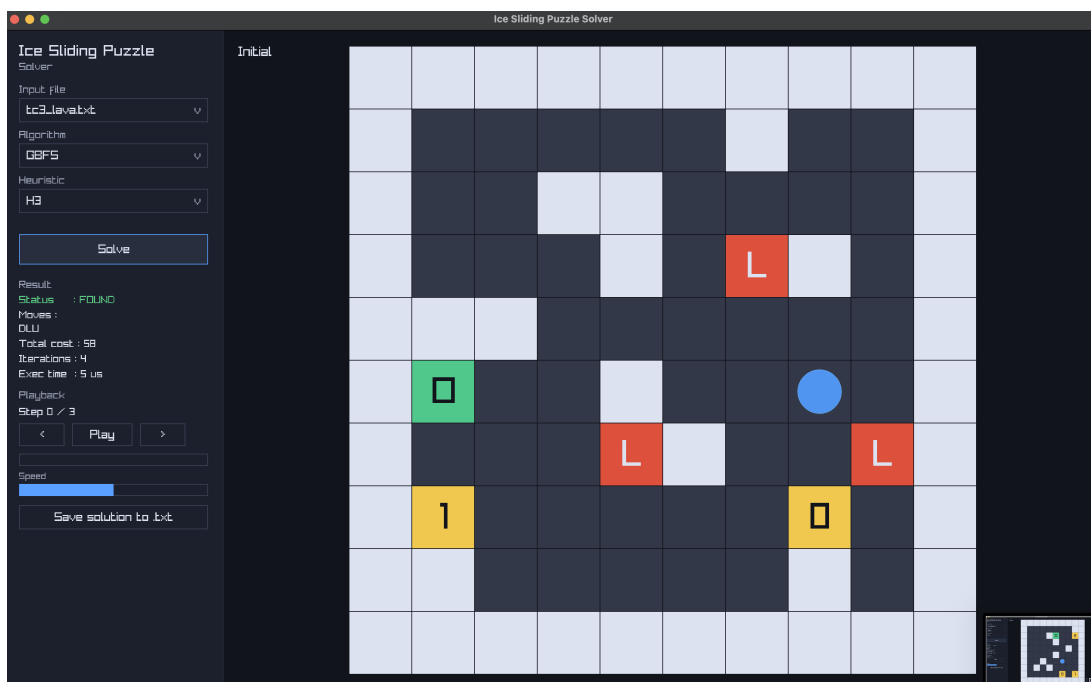
Gambar 5.15: UCS



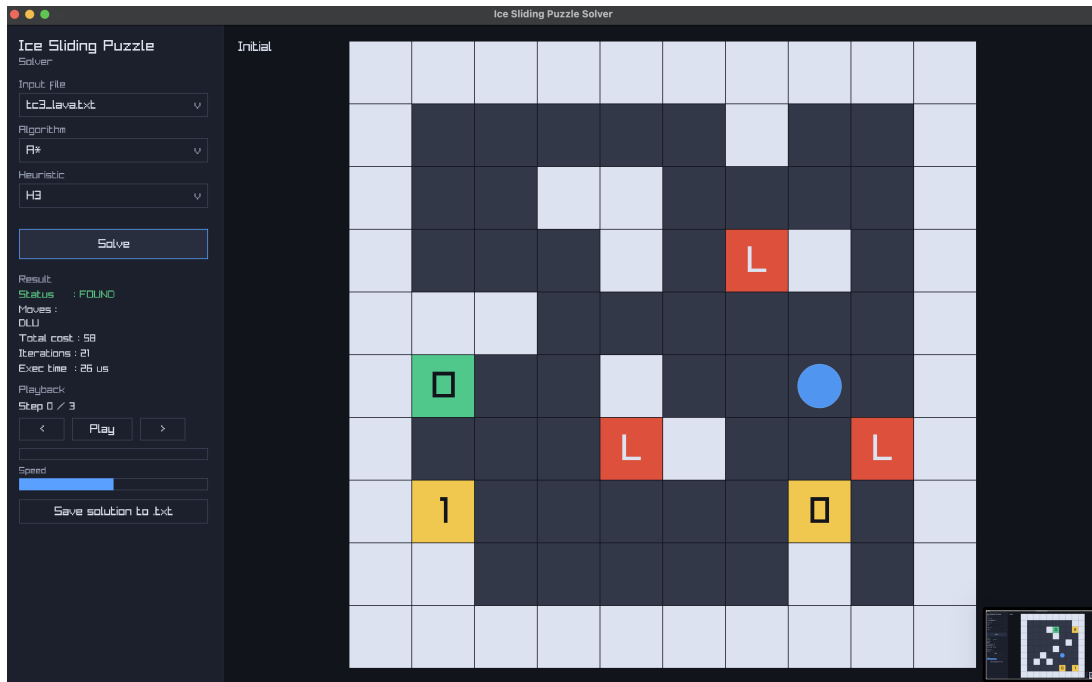
Gambar 5.16: GBFS H1



Gambar 5.17: GBFS H2



Gambar 5.18: GBFS H3



Gambar 5.21: A* H3

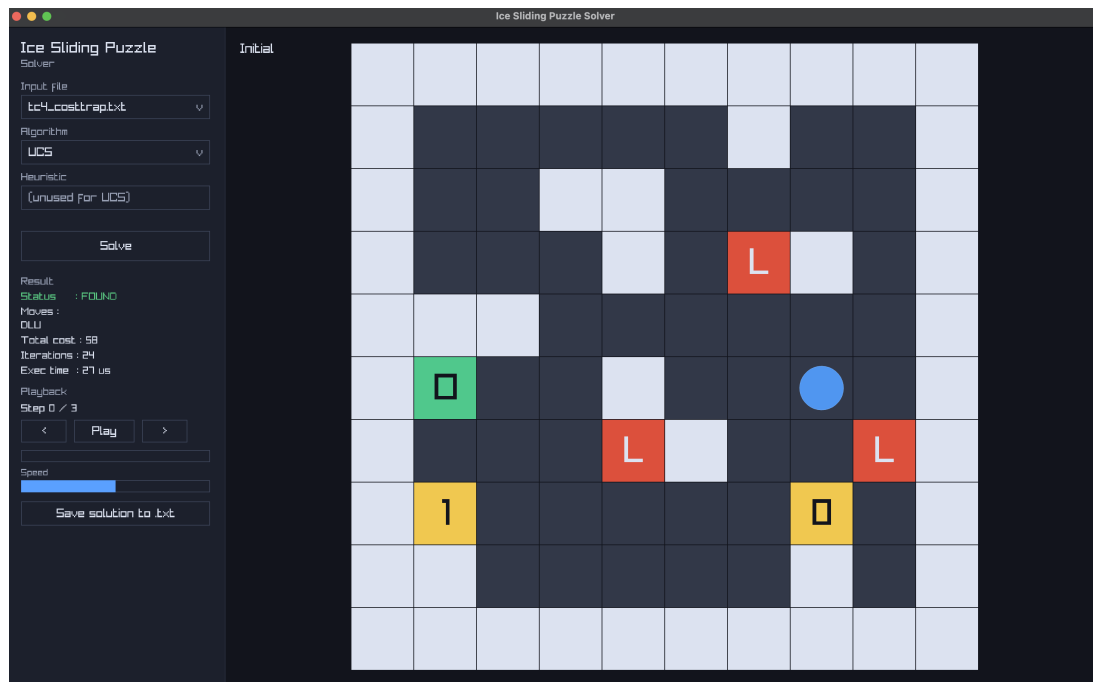
Tabel 5.4: Hasil TC-3

Algoritma	Solusi	Cost	Iterasi
UCS	DLU	58	24
GBFS H1	DLU	58	19
GBFS H2	DLU	58	9
GBFS H3	DLU	58	4
A* H1	DLU	58	21
A* H2	DLU	58	21
A* H3	DLU	58	21

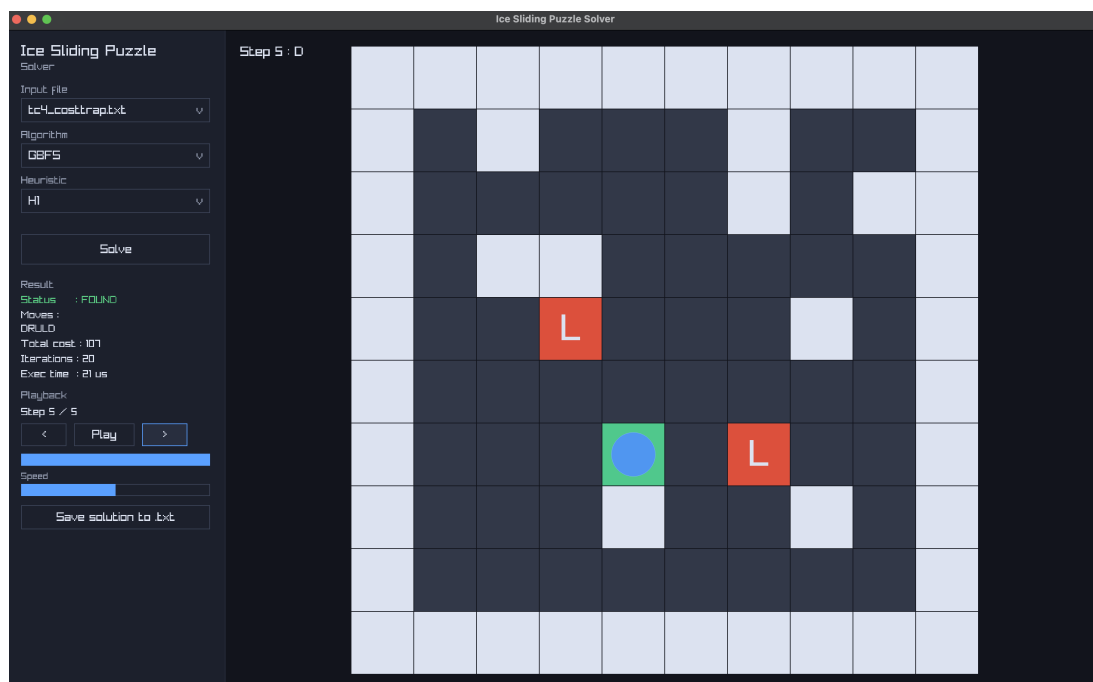
5.2.4 Test Case 4

Test case ini menggunakan file `test/tc4_costttrap.txt` berukuran 10x10. Tata letak dirancang untuk memperlihatkan jebakan biaya pada jalur yang tampak dekat dengan goal.

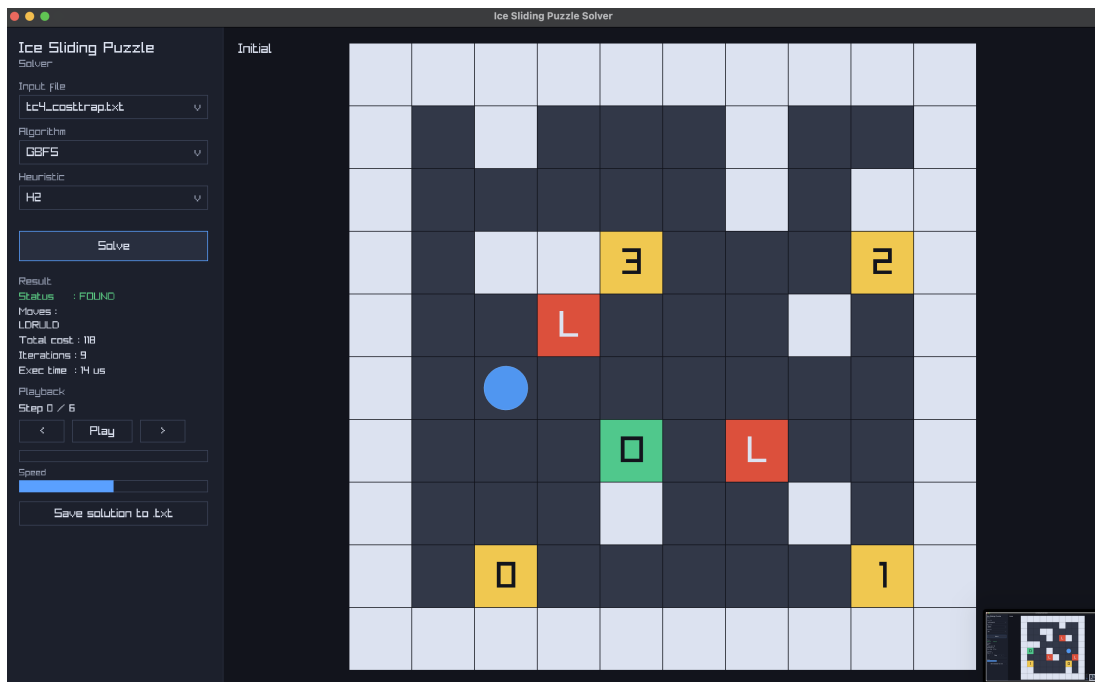
Perbandingan TC-4 menyortir perbedaan keputusan antara algoritma greedy dan algoritma berbasis biaya.



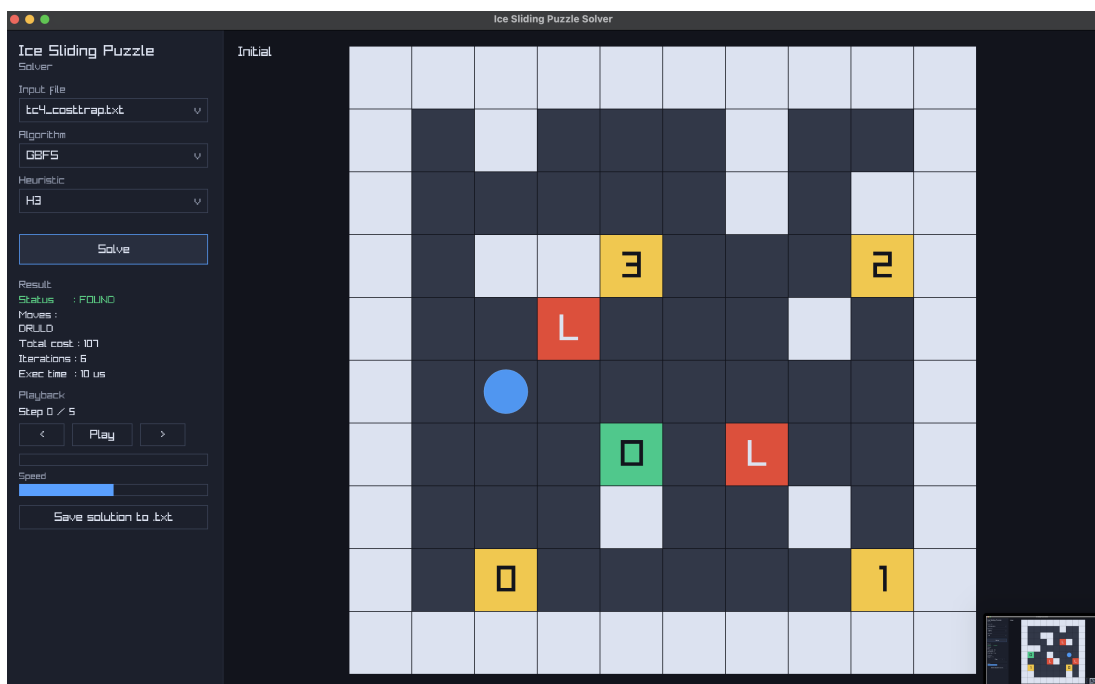
Gambar 5.22: UCS



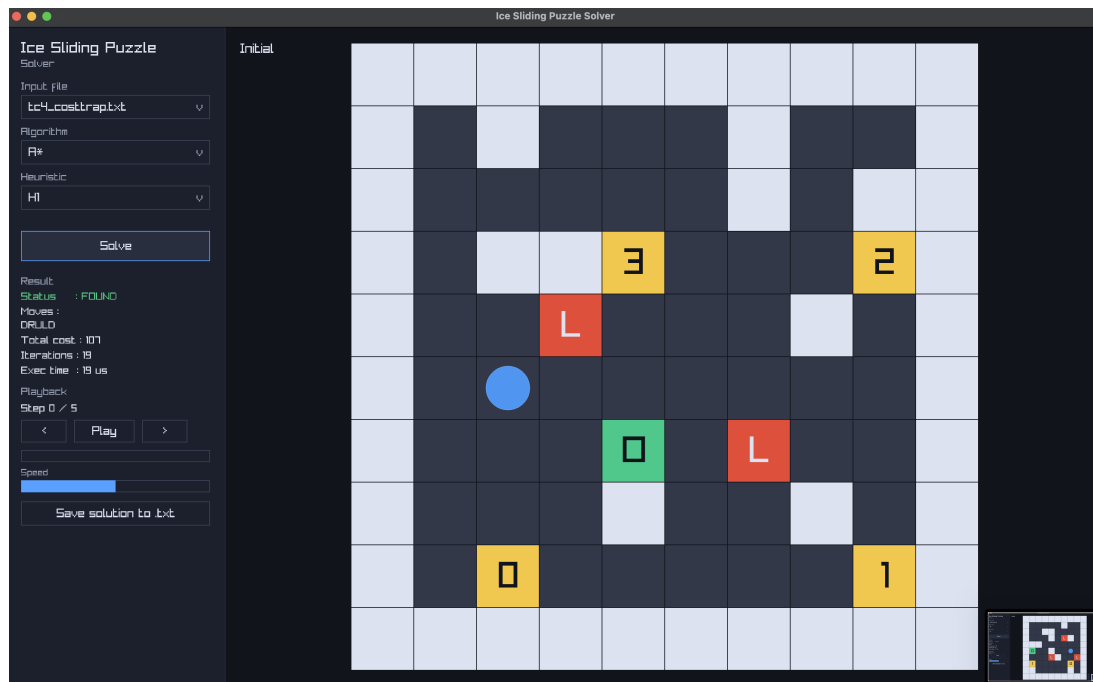
Gambar 5.23: GBFS H1



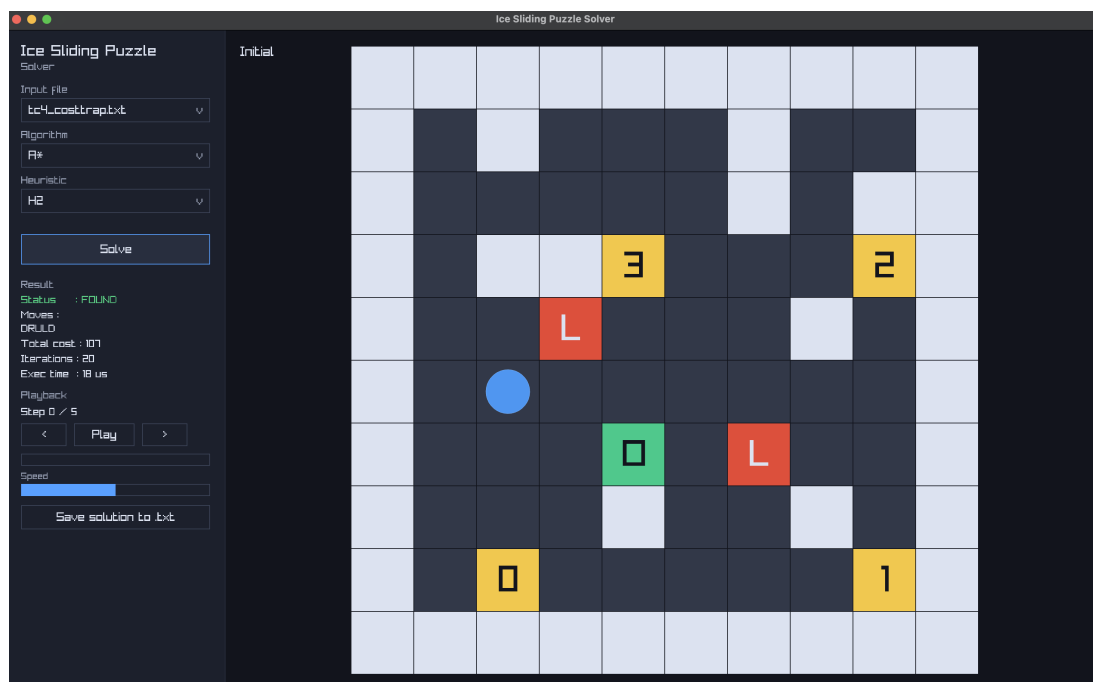
Gambar 5.24: GBFS H2



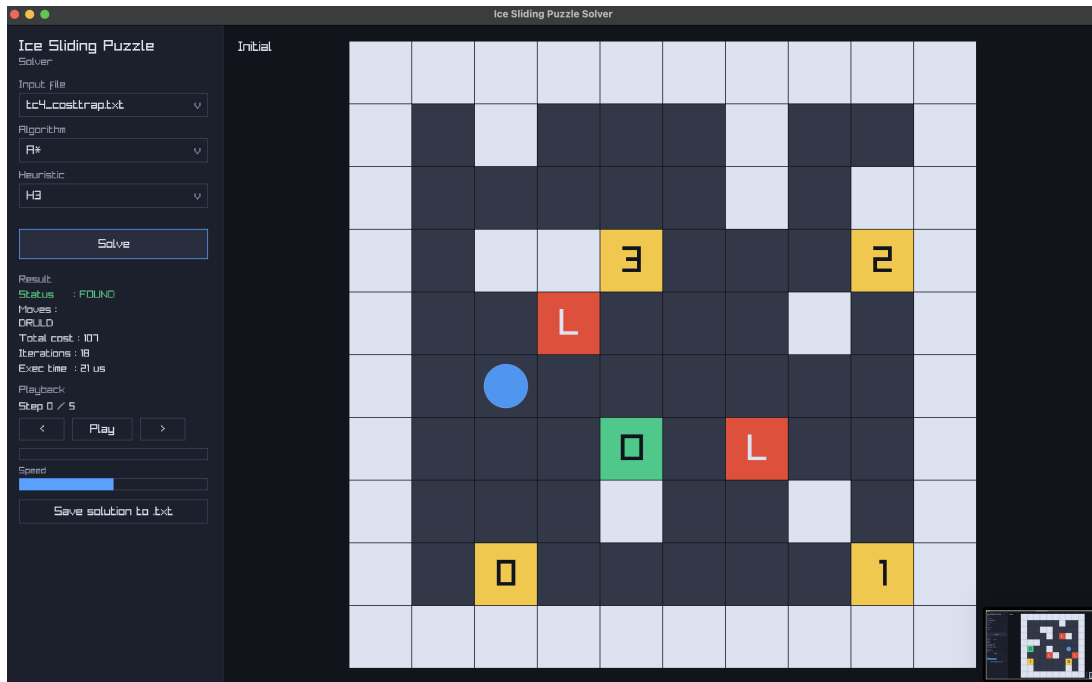
Gambar 5.25: GBFS H3



Gambar 5.26: A* H1



Gambar 5.27: A* H2



Gambar 5.28: A* H3

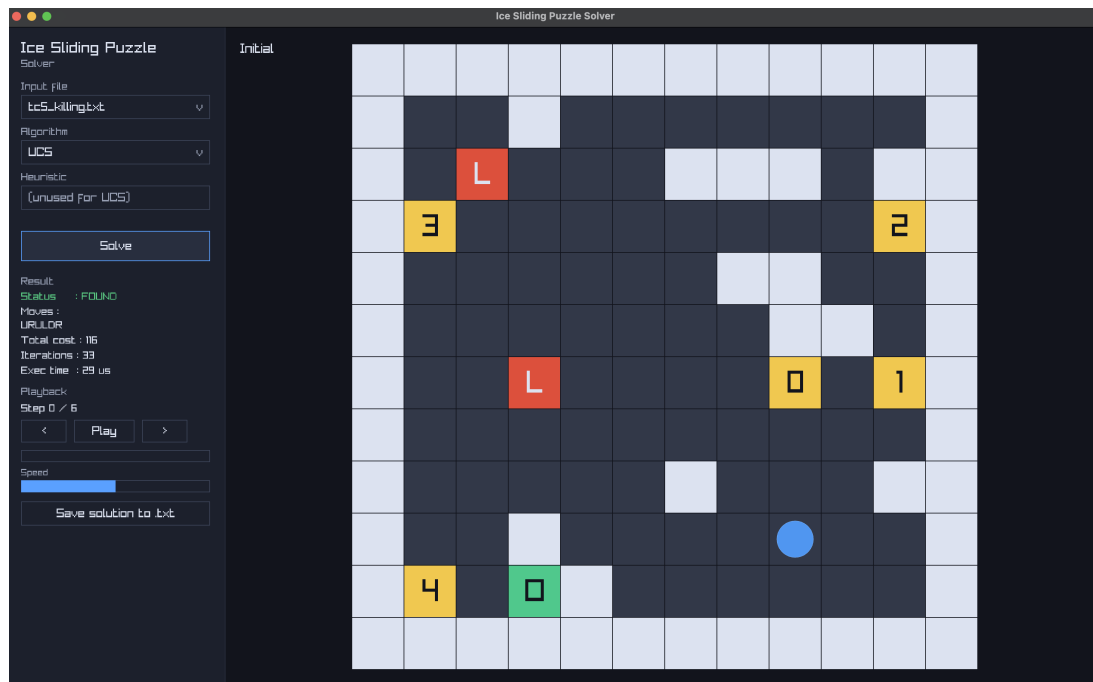
Tabel 5.5: Hasil TC-4

Algoritma	Solusi	Cost	Iterasi
UCS	DRULD	107	23
GBFS H1	DRULD	107	20
GBFS H2	LDRULD	118	9
GBFS H3	DRULD	107	6
A* H1	DRULD	107	19
A* H2	DRULD	107	20
A* H3	DRULD	107	18

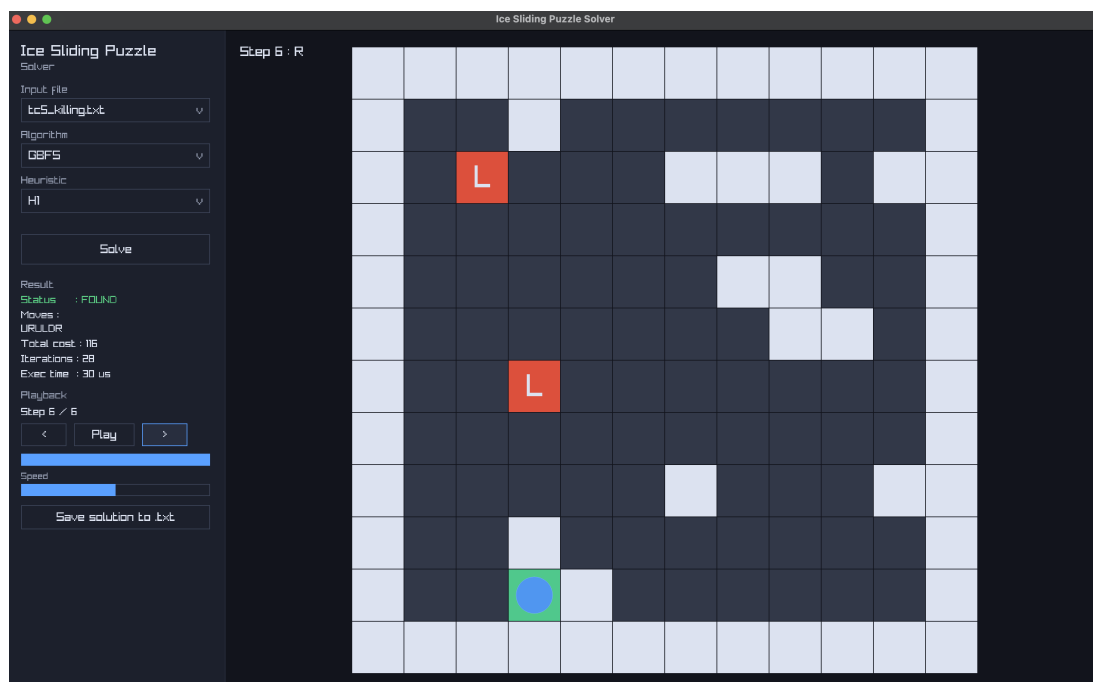
5.2.5 Test Case 5

Test case ini menggunakan file `test/tc5_killing.txt` berukuran 12x12 dan memiliki lava. Ini merupakan skenario yang lebih menantang dibanding TC-1 hingga TC-4.

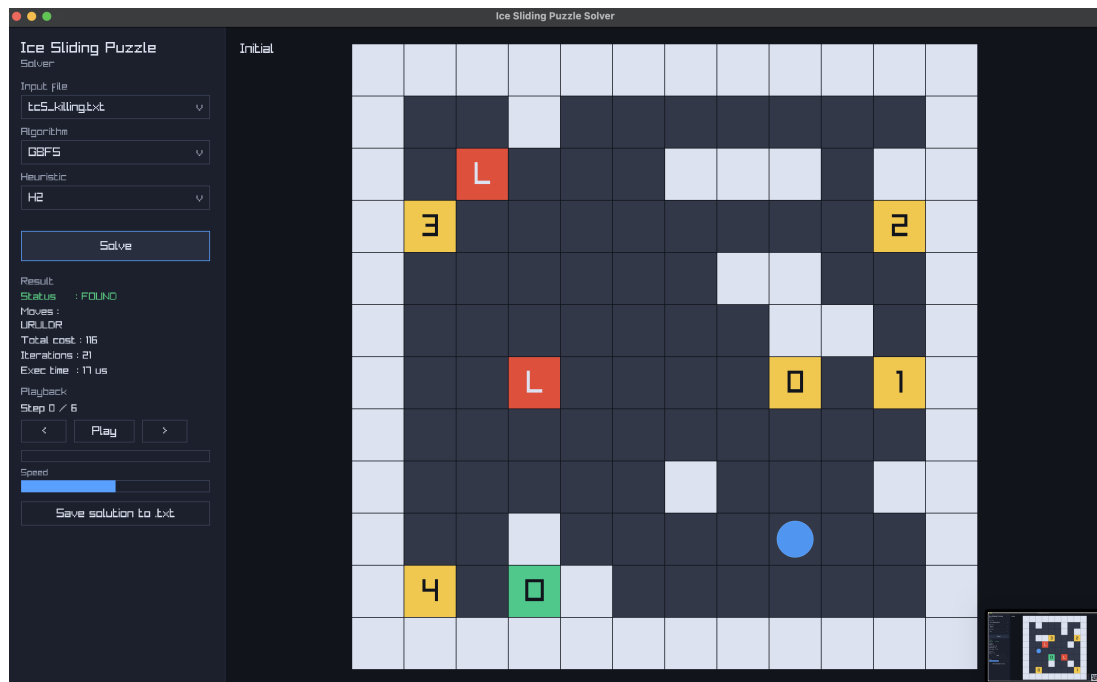
TC-5 digunakan untuk mengamati efek kombinasi ukuran papan dan lava terhadap jumlah iterasi serta kualitas solusi.



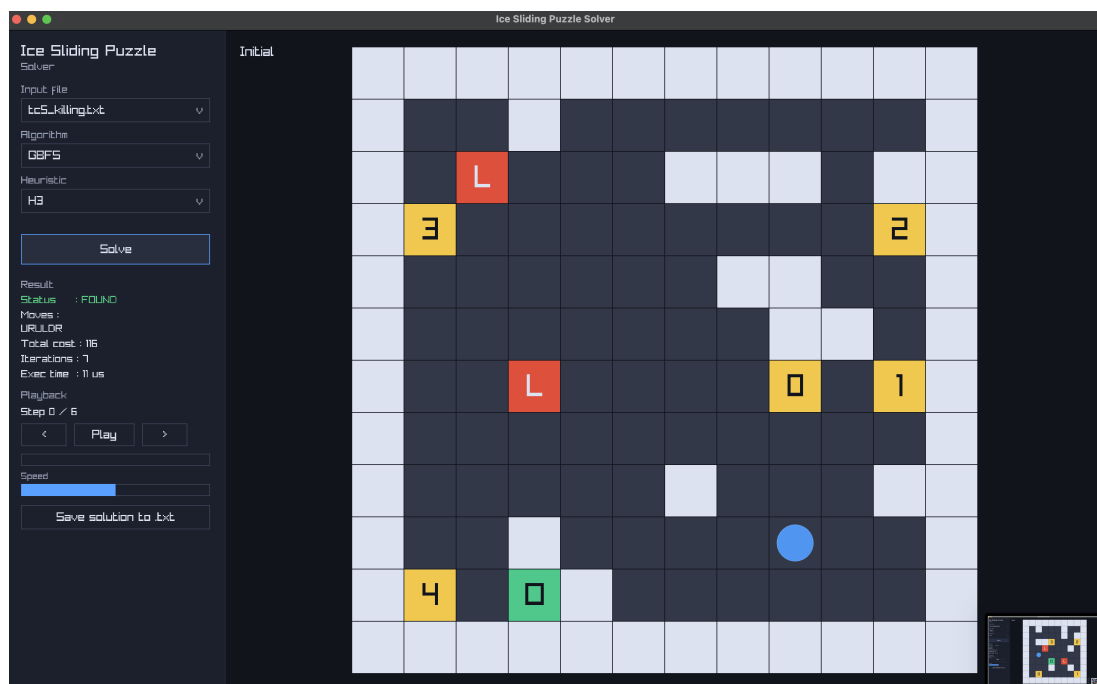
Gambar 5.29: UCS



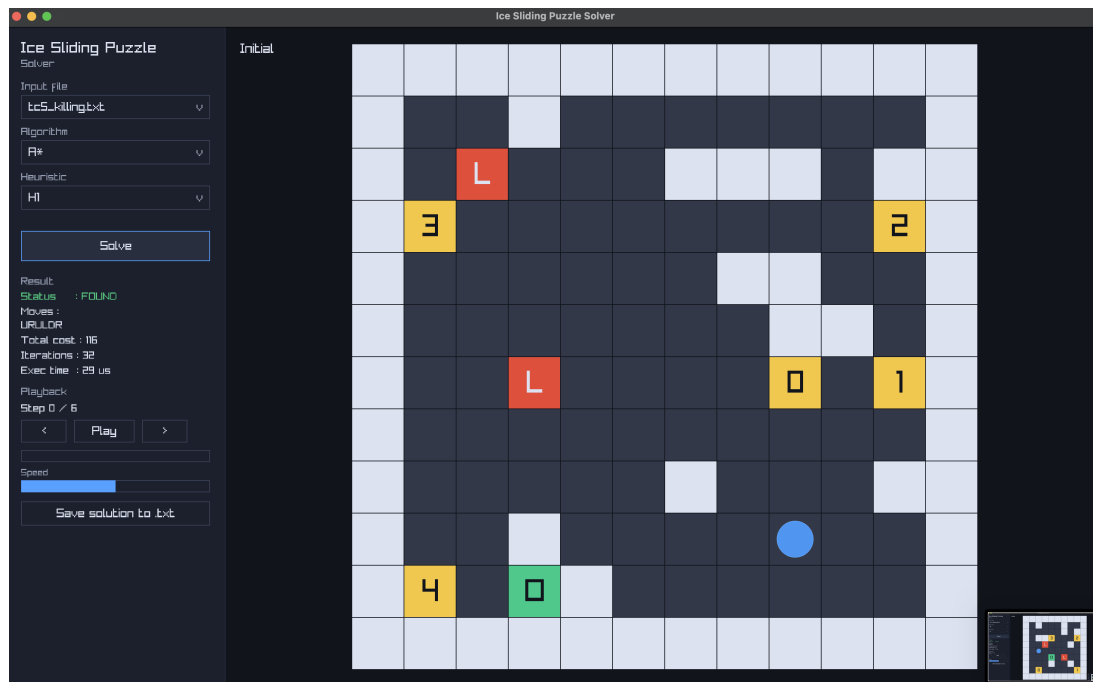
Gambar 5.30: GBFS H1



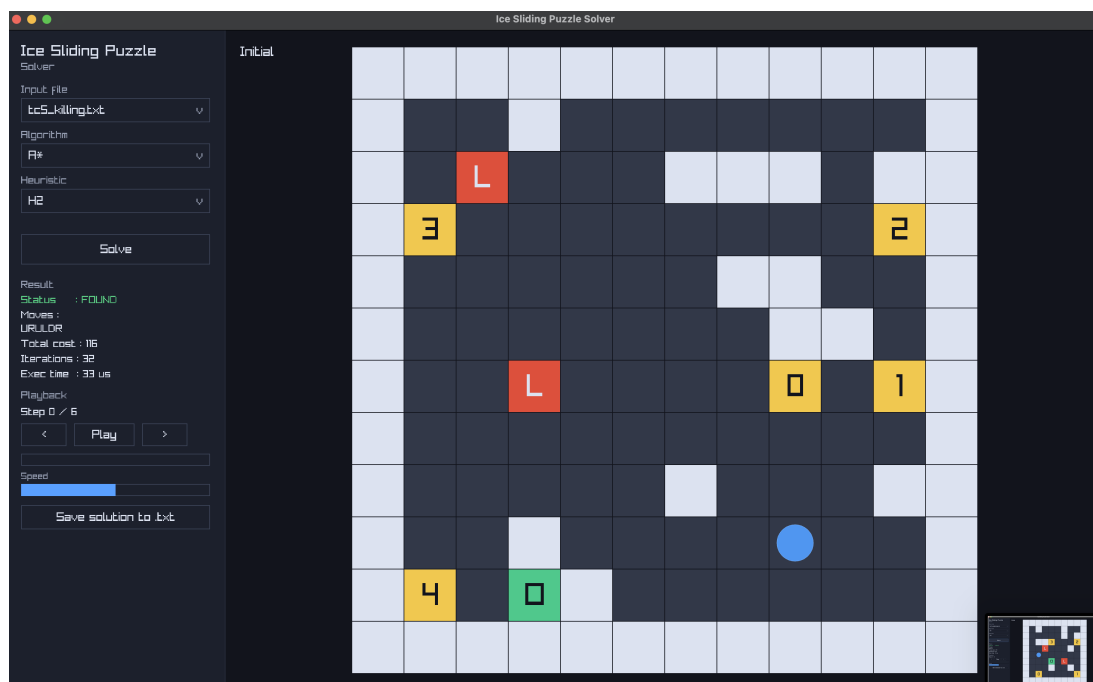
Gambar 5.31: GBFS H2



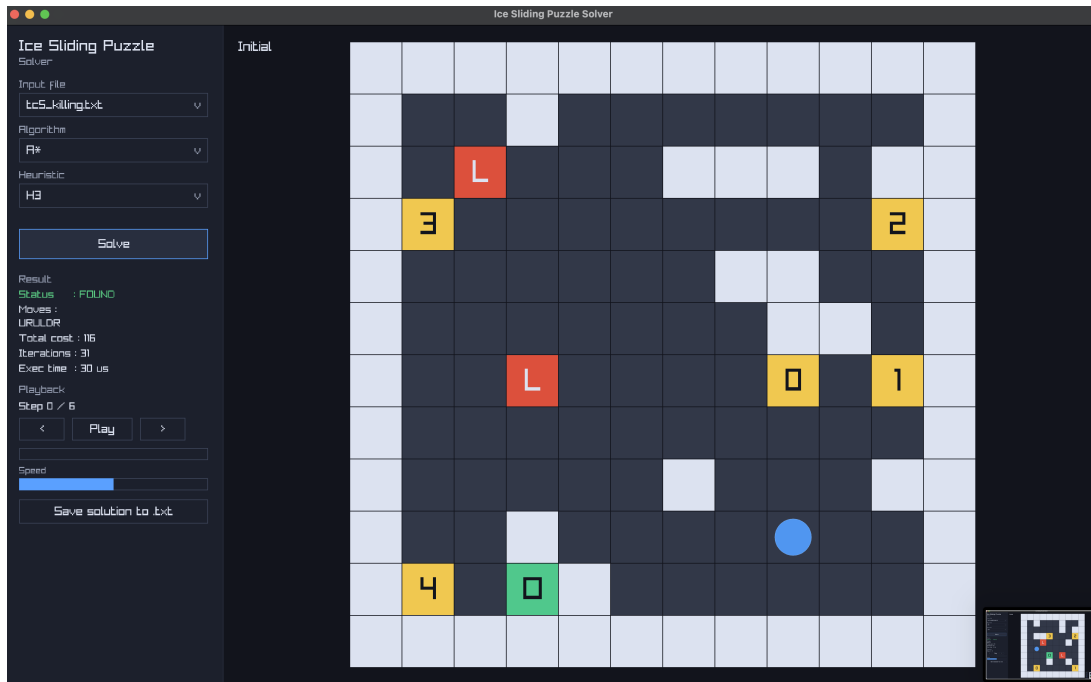
Gambar 5.32: GBFS H3



Gambar 5.33: A* H1



Gambar 5.34: A* H2



Gambar 5.35: A* H3

Tabel 5.6: Hasil TC-5

Algoritma	Solusi	Cost	Iterasi
UCS	URULDR	116	33
GBFS H1	URULDR	116	28
GBFS H2	URULDR	116	21
GBFS H3	URULDR	116	7
A* H1	URULDR	116	32
A* H2	URULDR	116	32
A* H3	URULDR	116	31

5.3 Analisis Hasil

Heuristik H1 berfokus pada jarak Manhattan ke target langsung, H2 memperhitungkan checkpoint secara berurutan, sedangkan H3 mengalikan H2 dengan biaya minimum untuk menghasilkan estimasi biaya yang lebih ketat.

Pada GBFS, perbedaan heuristik terutama memengaruhi kecepatan menemukan solusi karena algoritma tidak menggunakan biaya aktual. Pada A*, variasi heuristik memengaruhi jumlah iterasi dan waktu eksekusi, namun tetap mempertahankan optimalitas selama heuristik admissible.

Secara umum, A* diharapkan menghasilkan cost optimal ketika heuristik bersifat admissible, sedangkan GBFS tidak memberikan jaminan optimalitas. Pada papan kecil, perbedaan cost mungkin kecil, tetapi pada papan besar, perbedaan tersebut biasanya lebih terlihat.

GBFS sering memiliki iterasi lebih sedikit karena langsung mengarah ke target, namun bisa tersesat pada jalur mahal. UCS cenderung memiliki iterasi paling besar, tetapi

konsisten dalam kualitas solusi. Ukuran papan dan jumlah checkpoint meningkatkan kompleksitas ruang pencarian, sehingga manfaat heuristik menjadi semakin penting pada skala besar.

Bab 6

Kesimpulan

Tiga algoritma pencarian berhasil diimplementasikan dan mampu menyelesaikan Ice Sliding Puzzle sesuai aturan sliding, checkpoint, dan biaya. UCS memberikan solusi optimal dengan biaya eksplorasi yang besar, sedangkan A* menyeimbangkan optimalitas dan efisiensi berkat fungsi evaluasi yang menggabungkan $g(n)$ dan $h(n)$.

GBFS menunjukkan performa cepat pada banyak kasus, tetapi kualitas solusi tidak selalu optimal karena mengabaikan biaya aktual. Hasil eksperimen menunjukkan bahwa pemilihan heuristik dan ukuran papan sangat memengaruhi jumlah iterasi serta waktu eksekusi, sehingga pendekatan heuristik menjadi semakin penting pada papan besar.

Dari analisis hasil uji, terlihat bahwa variasi heuristik mempengaruhi perilaku pencarian secara signifikan. Heuristik yang hanya mengandalkan jarak langsung (H1) cenderung menghasilkan eksplorasi yang lebih agresif dan kadang mengarah pada solusi suboptimal pada skenario dengan jebakan biaya. Heuristik berbasis checkpoint (H2) memberikan arah pencarian yang lebih stabil karena mempertimbangkan urutan target yang wajib dilalui, sedangkan H3 memberikan estimasi biaya yang lebih ketat ketika biaya tile bervariasi, sehingga membantu memperkecil jumlah iterasi.

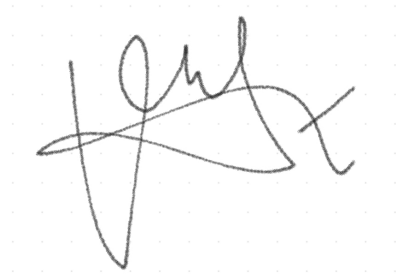
Secara umum, A* konsisten menghasilkan solusi optimal dengan iterasi lebih rendah dibanding UCS, terutama pada test case dengan lava dan jalur yang sempit. GBFS sering lebih cepat dalam menemukan solusi awal, tetapi kualitasnya sensitif terhadap jenis heuristik dan struktur papan. Temuan ini menegaskan bahwa trade-off antara kecepatan dan optimalitas sangat bergantung pada kualitas heuristik dan karakteristik test case.

Pernyataan Ketidakcurangan

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (*Generative AI*), melainkan hasil pemikiran dan analisis mandiri.

A handwritten signature in white ink on a black rectangular background. The signature is stylized, starting with a large 'S' followed by 'Ag' and ending with 'Wongso'.

Stevanus Agustaf Wongso

A handwritten signature in black ink on a light gray dotted grid background. The signature is highly stylized, featuring a large 'V' and 'R'.

Vincent Rionarlie

Bibliografi

- [1] Russell, S., & Norvig, P. (2010). *Artificial Intelligence: A Modern Approach* (3rd ed.). Prentice Hall.
- [2] Hart, P. E., Nilsson, N. J., & Raphael, B. (1968). A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics*, 4(2), 100–107.
- [3] Dijkstra, E. W. (1959). A note on two problems in connexion with graphs. *Numerische Mathematik*, 1(1), 269–271.
- [4] Raylib Contributors. (2024). *raylib: A simple and easy-to-use library to enjoy video-games programming*. <https://www.raylib.com>
- [5] Munir, R. (2026). *Route Planning (2026) Bagian 1*. [https://informatika.stei.itb.ac.id/~rinaldi.munir/Stmik/2025-2026/21-Route-Planning-\(2026\)-Bagian1.pdf](https://informatika.stei.itb.ac.id/~rinaldi.munir/Stmik/2025-2026/21-Route-Planning-(2026)-Bagian1.pdf)
- [6] Munir, R. (2026). *Route Planning (2026) Bagian 2*. [https://informatika.stei.itb.ac.id/~rinaldi.munir/Stmik/2025-2026/22-Route-Planning-\(2026\)-Bagian2.pdf](https://informatika.stei.itb.ac.id/~rinaldi.munir/Stmik/2025-2026/22-Route-Planning-(2026)-Bagian2.pdf)

Lampiran A

Tabel Checklist

Lampiran ini berisi tabel checklist untuk memastikan seluruh poin spesifikasi terpenuhi sebelum pengumpulan. Dengan checklist ini, proses verifikasi menjadi terstruktur dan mengurangi risiko ada poin yang terlewat.

Nilai Ya/Tidak dapat diperbarui sesuai kondisi aktual implementasi, termasuk keberadaan GUI, algoritma tambahan, dan variasi heuristik.

Tabel A.1: Tabel Checklist Pemenuhan Spesifikasi

No	Poin	Ya	Tidak
1	Program berhasil dikompilasi tanpa kesalahan	[✓/—]	
2	Program berhasil dijalankan	[✓/—]	
3	Solusi yang diberikan program benar dan mematuhi aturan permainan	[✓/—]	
4	Program dapat membaca masukan berkas <code>.txt</code> serta menyimpan solusi dalam berkas <code>.txt</code>	[✓/—]	
5	Program memiliki <i>Graphical User Interface</i> (GUI)	[✓/—]	
6	Ada algoritma pathfinding alternatif selain dari spesifikasi wajib		[✓/—]
7	Ada tambahan dua alternatif heuristik selain yang utama	[✓/—]	

Lampiran B

Pranala Repository

Repository program tersedia secara publik pada tautan berikut:

https://github.com/Vixrlie/Tucil3_13524020_13524031